

RESEARCH

Open Access



SSLA: a semi-supervised framework for real-time injection detection and anomaly monitoring in cloud-based web applications with real-world implementation and evaluation

Seyed Salar Sefati^{1,2,3*}, Bahman Arasteh^{3,4,5}, Octavian Fratu^{1,2,6} and Simona Halunga^{1,2,6}

Abstract

Injection attacks and anomalies pose significant threats to the security and reliability of cloud-based web applications. Traditional detection methods, such as rule-based systems and supervised learning techniques, often struggle to adapt to evolving threats and large-scale, unstructured log data. This paper introduces a novel framework, the Semi-Supervised Log Analyzer (SSLA), designed for real-time injection detection and anomaly monitoring in cloud environments. SSLA uses semi-supervised learning to utilize both labeled and unlabeled data, reducing the reliance on extensive annotated datasets. A similarity graph is built from the log data, allowing for effective anomaly detection using graph-based methods. At the same time, privacy-preserving techniques are integrated to protect sensitive information. The proposed method is evaluated on large-scale datasets, including Hadoop Distributed File System (HDFS) and BlueGene/L (BGL) logs, demonstrating superior performance in terms of precision, recall, and scalability compared to state-of-the-art methods. SSLA achieves high detection accuracy with minimal computational overhead, ensuring reliable, real-time protection for cloud-based web applications.

Keywords Anomaly detection, Cloud computing, Web applications, Semi-Supervised framework

*Correspondence:

Seyed Salar Sefati
Sefati.seyedsalar@upb.ro

¹Telecommunications Department, Faculty of Electronics, Telecommunications and Information Technology, National University of Science and Technology POLITEHNICA Bucharest, Bucharest 060042, Romania

²Research Center Campus, POLITEHNICA Bucharest, Bucharest 060042, Romania

³Department of Software Engineering, Faculty of Engineering and Natural Science, Istinye University, Istanbul 34460, Türkiye

⁴Department of Computer Science, Khazar University, Baku AZ1096, Azerbaijan

⁵Applied Science Research Center, Applied Science Private University, Amman 11937, Jordan

⁶Academy of Romanian Scientists, Bucharest 050044, Romania

Introduction

Cloud-based web applications are software systems that utilize cloud computing infrastructure to deliver services over the internet, enabling scalability, flexibility, and high availability to meet diverse user demands and large-scale data processing [1]. These applications reduce operational costs, enhance resource efficiency, and ensure accessibility across multiple devices and locations by using cloud platforms [2]. The adoption of advanced technologies, such as microservices, containerization, and serverless computing, further improves their performance and reliability [3]. However, the distributed and interconnected

nature of cloud-based web applications increases their vulnerability to security threats, particularly injection attacks and anomalies [4]. Developing robust detection mechanisms is essential for protecting sensitive data from breaches and maintaining continuous service availability. This is particularly critical in environments with strict privacy regulations and high security demands [5]. Injection detection and anomaly monitoring are important processes for identifying and mitigating security threats and irregularities in web applications [6].

Injection attacks, such as SQL injection [7], cross-site scripting (XSS) [8], and command injection, exploit vulnerabilities in application code to execute malicious commands, leading to data breaches, unauthorized access, and service disruptions [9]. Anomaly monitoring, on the other hand, focuses on detecting deviations from normal system behavior, which may indicate potential security threats or performance issues [10]. Together, these mechanisms safeguard applications by continuously analyzing system inputs, user behaviors, and network patterns using advanced techniques, such as machine learning, statistical models, and graph-based methods. Integrating privacy-preserving approaches ensures that sensitive data remains secure while enabling accurate detection in cloud-based environments [11]. Dealing with these challenges is important to keeping modern web applications secure, reliable, and running smoothly, especially in large-scale cloud environments.

In this paper, a novel method Semi-Supervised Log Analyzer (SSLA) is developed for real-time injection detection and anomaly monitoring in cloud-based web applications. At first, raw logs are preprocessed to extract meaningful features by normalizing, tokenizing, and filtering unstructured log entries into a structured format suitable for analysis. Then, statistical features such as mean, variance, skewness, and kurtosis are computed, alongside temporal features derived from log inter-arrival times, to capture patterns in system behavior. This method uses a semi-supervised learning model based on a Graph Convolutional Network (GCN) [12] which is trained to use both labeled and unlabeled data. Finally, the trained model assigns anomaly scores to log entries, enabling the identification of injection attacks and abnormal behaviors in a privacy-preserving and efficient manner. Contributions of the paper are:

- SSLA: This paper introduces SSLA, a novel framework that uses semi-supervised learning and graph-based techniques to detect injection attacks and anomalies in cloud-based web applications. The framework uses both labeled and unlabeled data,

reducing the need for large annotated datasets and improving detection accuracy.

- Integration of Privacy-Preserving Mechanisms: A key innovation of SSLA is the integration of privacy-preserving mechanisms into the anomaly detection pipeline. Incorporating differential privacy into the graph construction and label propagation processes enables SSLA to protect sensitive system and user data without compromising detection performance.
- Comprehensive Evaluation on Real-World Datasets: The SSLA framework is validated using large-scale datasets, including Hadoop Distributed File System (HDFS) [13] and BlueGene/L (BGL) logs [14]. Experimental results demonstrate that SSLA outperforms existing methods in terms of precision, recall, scalability, and computational efficiency, ensuring robust real-time anomaly detection in dynamic cloud environments.
- Quality of Service (QoS)-Aware Anomaly Detection: SSLA is designed with QoS considerations, optimizing key performance metrics such as detection latency, throughput, resource utilization, and availability. The framework ensures high anomaly detection accuracy while maintaining minimal impact on system performance, making it suitable for real-time applications in resource-constrained cloud environments.

The paper is organized as follows: Sect. 2 provides background on injection attacks, their impact, and the role of semi-supervised and graph-based methods in anomaly detection. Section 3 reviews related work, covering injection detection techniques, anomaly monitoring approaches, and graph-based methods for log analysis, followed by the motivation for this research. Section 4 outlines the problem statement, introduces key log terminology, discusses graph-based anomaly detection using semi-supervised learning, explores QoS considerations, and details the proposed method. Section 5 presents the experimental results, including datasets, evaluation metrics, comparison algorithms, and the performance evaluation of anomaly detection methods, with a focus on QoS analysis for HDFS and BGL datasets. Section 6 discusses the results, offering architectural insights and performance rationalization. Section 7 highlights open issues and challenges for future exploration. Finally, Sect. 8 concludes the paper and suggests potential directions for future research.

Background

The detection of injection attacks and anomalies has historically relied on rule-based systems and statistical methodologies [15]. These approaches, while

foundational, exhibit critical limitations when applied to the dynamic and large-scale nature of cloud-based web applications. Rule-based systems depend on predefined patterns to identify vulnerabilities exploited in injection attacks, such as SQL injection and XSS [16]. However, they suffer from high false-positive rates and fail to generalize to novel or obfuscated attack strategies. Statistical models, which aim to capture deviations from expected behavior, require extensive domain expertise to establish thresholds and are inherently limited in their ability to process the unstructured nature of log data. Consequently, these traditional techniques lack the scalability and adaptability necessary for safeguarding modern, distributed cloud infrastructures.

Injection attacks and their impact

Injection attacks are a critical security threat to cloud-based web applications, enabling attackers to exploit vulnerabilities in input validation mechanisms and manipulate system queries or commands. Among these, SQL injection (SQLi) and XSS are particularly pervasive [17]. SQL injection targets backend databases by injecting malicious SQL statements into input fields that are improperly sanitized. For example, a vulnerable SQL query such as *SELECT * FROM users WHERE username=' username' AND password='password'* can be exploited by supplying inputs such as *' OR '1'='1* to bypass authentication. Advanced forms of SQLi, including union-based, blind, and time-based SQLi, further enable attackers to extract sensitive data, modify records, or execute administrative commands. These techniques exploit the logic and structure of SQL queries to gain unauthorized access to critical resources, highlighting the importance of parameterized queries and secure coding practices [18]. XSS attacks, on the other hand, exploit the trust between a web application and its users by injecting malicious scripts into webpages that are later rendered in the victim's browser [19, 20]. XSS attacks are categorized as stored, reflected, or DOM-based. Stored XSS involves the permanent storage of malicious scripts on the server, which are executed whenever affected pages are loaded by users. Reflected XSS occurs when the malicious payload is embedded in URLs or request parameters and immediately executed upon being reflected in the response. DOM-based XSS manipulates client-side scripts to execute injected payloads by exploiting vulnerabilities in JavaScript code. These attacks allow adversaries to steal session cookies, impersonate users, deface websites, or distribute malware, posing a significant risk to application integrity and user privacy [21]. The impact

of these attacks is far-reaching, with SQLi compromising backend data integrity and confidentiality, while XSS undermines user trust and application functionality. In cloud-based environments, where distributed architectures are prevalent, the risks are amplified as vulnerabilities in one service can propagate across interconnected systems. These challenges require a multifaceted approach, including robust input validation, secure coding practices, adoption of parameterized queries, and implementation of Content Security Policies (CSP) [22].

Semi-supervised and graph-based methods

Semi-supervised learning is a valuable approach for situations where labeled data is limited, but large amounts of unlabeled data are available [23]. Supervised methods that depend entirely on labeled samples, semi-supervised techniques extract patterns from unlabeled data to enhance model performance [24, 25]. Graph-based methods further enhance this capability by modeling data as nodes in a graph and their relationships as weighted edges. This synergy between semi-supervised learning and graph representation is particularly effective for tasks such as anomaly detection, where labeled anomalies are sparse and their patterns may evolve dynamically.

In a graph-based semi-supervised framework, a similarity graph $G = (V, E, W)$ is constructed, where V represents the data points, E denotes the edges between nodes, and W is the edge weight matrix encoding the similarity between connected nodes. Labels are propagated across the graph using iterative techniques, ensuring that nodes with high connectivity to labeled points receive consistent classifications. To improve robustness, differential privacy mechanisms can be incorporated into the graph construction and label propagation stages, safeguarding sensitive information. Additionally, spectral embedding techniques are applied to reduce the graph's dimensionality, retaining only the most relevant features for computational efficiency.

The pseudocode 1 outlines the general idea of a semi-supervised graph-based method for anomaly detection. This integration of semi-supervised and graph-based techniques enables scalable and interpretable anomaly detection in high-dimensional and dynamic environments. These methods use the graph's structure to effectively identify subtle anomalies that might be overlooked by traditional approaches. Furthermore, privacy-preserving mechanisms and dimensionality reduction ensure the approach is suitable for sensitive, large-scale datasets commonly encountered in cloud-based systems.

1. **Input:** Dataset $D = [x_1, x_2, \dots, x_N]$ labels $Y = [y_1, y_2, \dots, y_L]$ ($L \ll N$), similarity function $S(x_i, x_j)$, threshold δ
2. **Output:** Predicted labels $\hat{Y}$, anomaly scores $S(x)$
3. Construct a similarity graph $G = (V, E, W)$ using $S(x_i, x_j)$
4. **For all** nodes i in G **do**
5. Assign weights $W_{ij} = \exp(-\|x_i - x_j\|^2 / 2\sigma^2)$ if $S(x_i, x_j) > \delta$
6. **End for**
7. Apply spectral embedding to G , reducing dimensionality to k
8. Propagate labels Y across G using:

$$\hat{y}_i = \operatorname{argmin}_{\hat{y}_i} \sum_{(i,j) \in E} W_{ij} \cdot (\hat{y}_i - \hat{y}_j)^2$$

9. Compute anomaly scores for unlabeled nodes:

$$S(x_i) = 1 - \Pr(y_i = \text{normal} | x_i)$$

10. **return** $\hat{Y}, S(x)$

Algorithm 1. General idea of semi-supervised graph-based anomaly detection

Related work

This section reviews the existing literature in two main phases. The first phase discusses the state-of-the-art methods for injection detection and anomaly monitoring, including both traditional and modern machine learning-based approaches. The second phase examines advancements in graph-based methods and semi-supervised learning for log analysis, with a focus on addressing the challenges associated with unstructured and semi-structured data.

Injection detection and anomaly monitoring approaches

Injection detection and anomaly monitoring rely on rule-based and signature-based tools such as Snort and SIEM platforms. These systems use predefined patterns to identify and flag known threats. While effective for detecting established attack vectors like SQL injection and XSS, these methods struggle against novel or obfuscated attacks due to their static nature. Machine learning techniques, including supervised and unsupervised methods, have been applied to anomaly detection and behavior analysis to overcome these limitations [26]. These approaches detect anomalies by examining patterns in historical data and identifying deviations that could signal malicious activity [27]. However, they often depend on large labeled datasets, which are challenging and time-consuming to acquire.

Snort

Snort is one of the most widely deployed open-source intrusion detection systems (IDS) that uses signature-based detection to monitor network traffic for suspicious activities [28]. It operates by analyzing packets at the network level and comparing them against a comprehensive database of predefined rules to identify specific attack patterns, such as SQL injection or XSS. Snort's detection mechanism is implemented through a combination of pattern matching and protocol analysis. For instance, it

inspects HTTP headers, payloads, and query parameters to detect signs of malicious intent [29].

Bro/Zeek

Zeek, previously known as Bro, is an advanced network security monitoring and analysis framework that integrates both signature-based and behavior-based detection techniques [30]. Zeek offers a rich domain-specific scripting language that enables custom monitoring policies and in-depth protocol analysis. It allows for greater flexibility and visibility compared to fixed-rule IDS tools. It excels in detecting injection attacks by correlating network events, analyzing user behaviors, and identifying anomalies across multiple layers of the OSI model. For example, Zeek can detect SQL injection attempts by monitoring unusual SQL query patterns in HTTP traffic, even when payloads are encoded or fragmented [31].

Splunk

Splunk is a comprehensive SIEM platform that specializes in real-time log analysis, event correlation, and anomaly detection [32]. It ingests data from a wide range of sources, including application logs, network devices, and endpoint telemetry, and uses its proprietary search processing language (SPL) for querying and analysis [32, 33]. Splunk machine learning capabilities are particularly effective in identifying multi-stage injection attacks, such as those involving SQLi followed by privilege escalation or lateral movement.

Open source security event correlator (OSSEC)

OSSEC is a robust host-based intrusion detection system HIDS designed to monitor system-level activities, including log files, file integrity, and rootkit detection [34]. It is particularly suited for detecting injection attacks that exploit vulnerabilities in server-side applications, as it can analyze

error logs and detect unusual patterns in database queries or command executions [35]. OSSEC utilizes a multi-faceted detection approach, combining signature-based analysis with policy enforcement and real-time alerting. For instance, it can identify unauthorized changes to configuration files, which may indicate an ongoing injection attack.

Falco

Falco is a runtime security tool optimized for containerized and microservices-based environments [36]. It operates by monitoring system calls made by containers and processes, identifying abnormal behaviors indicative of injection attacks or other malicious activities. Falco uses a rich rule engine to detect anomalous patterns, such as unexpected file access, network connections, or command executions [37]. For example, it can flag suspicious attempts to execute shell commands within a container or access sensitive files outside the expected application context. Its real-time detection capabilities make it particularly effective for protecting container orchestration platforms like Kubernetes, where traditional IDS tools struggle to monitor ephemeral workloads. However, Falco accuracy heavily depends on well-defined policies, and its reliance on these rules can limit its adaptability to novel threats or complex multi-stage attacks. Table 1 compared the injection detection and anomaly monitoring tools.

Graph-based and semi-supervised methods for log analysis

Similarity graph construction

Logs are represented as nodes in a graph, where edges capture the similarity between log entries based on feature vectors. Edge weights are computed using metrics like cosine similarity, Gaussian kernels [38], or Jaccard coefficients [39] to quantify the relationships between logs. This graph serves as the foundation for anomaly detection and label propagation. Techniques that use dynamic graph construction allow continuous updates, enabling the detection of emerging patterns in real-time log streams.

Graph convolutional networks (GCNs)

GCNs use the graph structure to propagate information between nodes through iterative neighborhood aggregation. In log analysis, they use labeled log entries to learn feature embeddings for the entire graph, enabling classification or anomaly detection for unlabeled nodes [40]. Advanced

GCN variants, such as Graph Attention Networks (GATs), assign adaptive weights to edges, allowing the model to prioritize critical relationships in the graph [41].

Label propagation algorithms

Semi-supervised methods like label propagation iteratively spread labels from labeled nodes to unlabeled ones using edge weights as probabilities [42]. This approach is particularly effective in log analysis, where labeled data is sparse. Techniques such as harmonic functions and random walks on graphs enhance the robustness of label propagation by incorporating global graph structure.

Temporal graph modeling

Logs often exhibit sequential dependencies, which are captured using temporal graphs. These graphs model logs as nodes connected by temporal edges that represent their chronological order [43]. Temporal graph neural networks (TGNNs) [44] or dynamic GCNs analyze time-correlated events, enabling the detection of anomalies such as coordinated attacks or cascading system failures.

Dimensionality reduction in graphs

High-dimensional log features are reduced using techniques like spectral embeddings or Laplacian eigenmaps, which preserve the structural properties of the graph. These methods enable efficient processing of large-scale graphs while maintaining the integrity of relationships between log entries. Integrating dimensionality reduction with semi-supervised learning improves scalability and inference speed, especially in resource-constrained environments. Table 2 shows graph-based and semi-supervised methods for log analysis.

Variational autoencoders (VAEs) for anomaly detection

Recent studies have demonstrated the effectiveness of VAEs for unsupervised anomaly detection, particularly in domains such as industrial control systems, cybersecurity telemetry, and IoT sensor data [45]. VAEs extend autoencoders by learning a probabilistic latent space, which enables better modeling of uncertainty and data variability [46]. They have shown improved generalization and robustness to noise, achieving F1 scores above 90% in several benchmarks. However, their application to log-based anomaly detection remains limited due to the inherent complexity of semi-structured and textual log formats.

Table 1 Comparison of injection detection and anomaly monitoring tools

Tool	Input	Analysis Focus	Detection Method
Snort	Network Traffic	HTTP headers, payloads, query parameters	Signature-based pattern matching
Zeek (Bro)	Network Logs	Event correlation, multi-layer OSI analysis	Behavior and signature-based detection
Splunk	Logs and Event Data	Log correlation, multi-stage attack analysis	Machine learning and rule-based models
OSSEC	System Logs, Files	File integrity, rootkit detection, system anomalies	Signature-based and policy enforcement
Falco	System Calls, Containers	Container runtime, process-level anomaly detection	Rule-based real-time monitoring

Table 2 Graph-Based and Semi-Supervised methods for log analysis

Tool	Input	Analysis Focus	Detection Method
Similarity Graph Construction	Log Entries, Feature Vectors	Capturing relationships between log entries	Computes edge weights using cosine similarity, Gaussian kernels, or Jaccard index
Graph Convolutional Networks (GCNs)	Graph Structures	Label propagation, feature embedding	Iteratively aggregates node neighborhoods; GATs prioritize critical relationships
Label Propagation Algorithms	Labeled and Unlabeled Nodes	Spreading labels across the graph	Uses harmonic functions or random walks for robust propagation
Temporal Graph Modeling	Log Entries with Timestamps	Detecting time-correlated anomalies	Models temporal dependencies with TGNNs or dynamic GCNs
Dimensionality Reduction in Graphs	High-Dimensional Graph Structures	Reducing computational complexity while preserving relationships	Uses spectral embeddings or Laplacian eigenmaps
Variational Autoencoder (VAE)	Sensor data, ICS telemetry, cybersecurity logs	Modeling uncertainty and latent representations of normal behavior	Unsupervised anomaly detection via reconstruction loss and probabilistic latent space modeling

Motivation for this research

Existing anomaly detection methods for cloud-based web applications suffer from three major challenges: (1) high reliance on labeled data, which is scarce and expensive to generate; (2) inability to generalize to evolving or obfuscated injection attacks; and (3) lack of mechanisms to preserve data privacy in real-time log analysis. Rule-based and supervised learning methods often lack the scalability and robustness required for the dynamic, distributed nature of cloud environments. Their limitations hinder effective adaptation to evolving threats and complex system behaviors. Furthermore, many state-of-the-art models do not account for QoS constraints such as latency, resource usage, or energy efficiency, which are critical for practical deployment.

Advanced machine learning models were analyzed for their adaptability to real-world log data. Supervised methods, such as classification-based anomaly detectors, demonstrated high accuracy when large, well-labeled datasets were available. However, the scarcity of labeled injection-related data in production systems limited their applicability. In contrast, unsupervised methods, such as clustering or autoencoders, failed to achieve robust performance in scenarios with high variability in log formats or noise. Furthermore, these approaches exhibited high false-positive rates, especially in semi-structured and unstructured log datasets commonly found in distributed cloud architectures.

Graph-based approaches were also evaluated for their ability to capture the structural dependencies and temporal relationships inherent in log data. While these methods, such as similarity graph construction and label propagation algorithms, showed promise, they faced significant computational overhead when applied to large-scale datasets. Additionally, existing graph models lacked mechanisms to ensure adversarial robustness and privacy compliance, which are analytical in cloud environments dealing with sensitive user data. Temporal graph-based methods exhibited potential for modeling sequential injection attacks but required more scalable and interpretable solutions to handle real-time streaming logs.

Building on these findings, the proposed SSLA tackles these challenges using a hybrid approach that combines semi-supervised learning, privacy-preserving graph techniques, and robust label propagation. This integration enhances both the accuracy and resilience of anomaly detection in sensitive cloud environments. The SSLA employs a dynamic similarity graph construction process to continuously capture relationships between log entries, even in evolving datasets. Incorporating spectral embeddings alongside differential privacy mechanisms boosts both computational efficiency and data security. Incorporating spectral embeddings alongside differential privacy mechanisms boosts both computational efficiency and data security. The integration of GCNs strengthens the model's ability to effectively spread label information across labeled and unlabeled nodes.

Problem statement

Cloud-based web applications have become integral to modern digital infrastructure, offering scalability, flexibility, and high availability. However, their distributed and interconnected nature makes them highly susceptible to security threats, particularly injection attacks (e.g., SQL injection, cross-site scripting) and anomalies that can lead to data breaches, unauthorized access, and service disruptions. Rule-based and signature-based detection systems, while effective against known attack patterns, often struggle to adapt to novel or obfuscated threats. Their static nature limits their ability to detect emerging and sophisticated attack vectors [47]. Moreover, the reliance on labeled datasets for supervised learning approaches is often impractical due to the scarcity of annotated data in real-world scenarios. Unsupervised methods, on the other hand, suffer from high false-positive rates and lack the robustness needed to handle the dynamic and noisy nature of log data in cloud environments.

Graph-based and semi-supervised learning methods have shown potential in modeling the structure and timing of log data. However, they often struggle with key issues like resisting adversarial manipulation, safeguarding user privacy, and

maintaining computational efficiency. These limitations hinder their effectiveness in real-world, large-scale cloud environments. The lack of mechanisms to ensure data confidentiality during analysis and the inability to handle large-scale, real-time log streams further limit their applicability in cloud-based systems. Additionally, the integration of temporal features and secure label propagation in graph-based models remains underexplored, leaving a gap in the detection of coordinated and multi-stage injection attacks.

This paper introduces a new framework called SSLA that tackles real-time injection detection and anomaly monitoring in cloud-based web applications. It combines semi-supervised learning with privacy-aware graph techniques to handle limited labeled data and adversarial threats. The main challenge it addresses is building a system that's not only accurate and scalable but also protects user privacy during detection. The proposed solution aims to overcome the limitations of existing approaches by using advanced techniques such as probabilistic pseudo-labeling, secure graph construction, differential privacy, and spectral embedding, while ensuring computational efficiency and adaptability to dynamic cloud environments. Figure 1 illustrates the general architecture of anomaly detection in cloud-based web applications.

Log terminology

Logs in cloud-based web applications are analytical for tracking system operations, identifying anomalies, and

mitigating potential security threats. Figure 2 provides a comprehensive view of how raw log messages are processed within our proposed SSLA framework, transitioning from unstructured sequences to structured data that enhances anomaly detection capabilities.

Each log message consists of two primary components:

Log Header: The log header encapsulates essential metadata to contextualize each log entry, ensuring that system administrators and automated tools can accurately interpret and correlate logs:

- **Timestamp:** Indicates the precise moment an event occurred within the system. For example, 203,615 reflects the timestamp when the corresponding event took place.
- **Verbosity Level:** Represents the importance or severity of the logged event. Typical verbosity levels include *INFO* for routine operations, *WARNING* for potential issues, and *ERROR* for critical failures. In Fig. 1, the verbosity level *INFO* highlights that the messages pertain to standard, non-critical system activities.
- **Component:** Identifies the specific module or service that generated the log message. For instance, Responder is responsible for handling packet responses, while Block Scanner oversees the verification of data blocks.

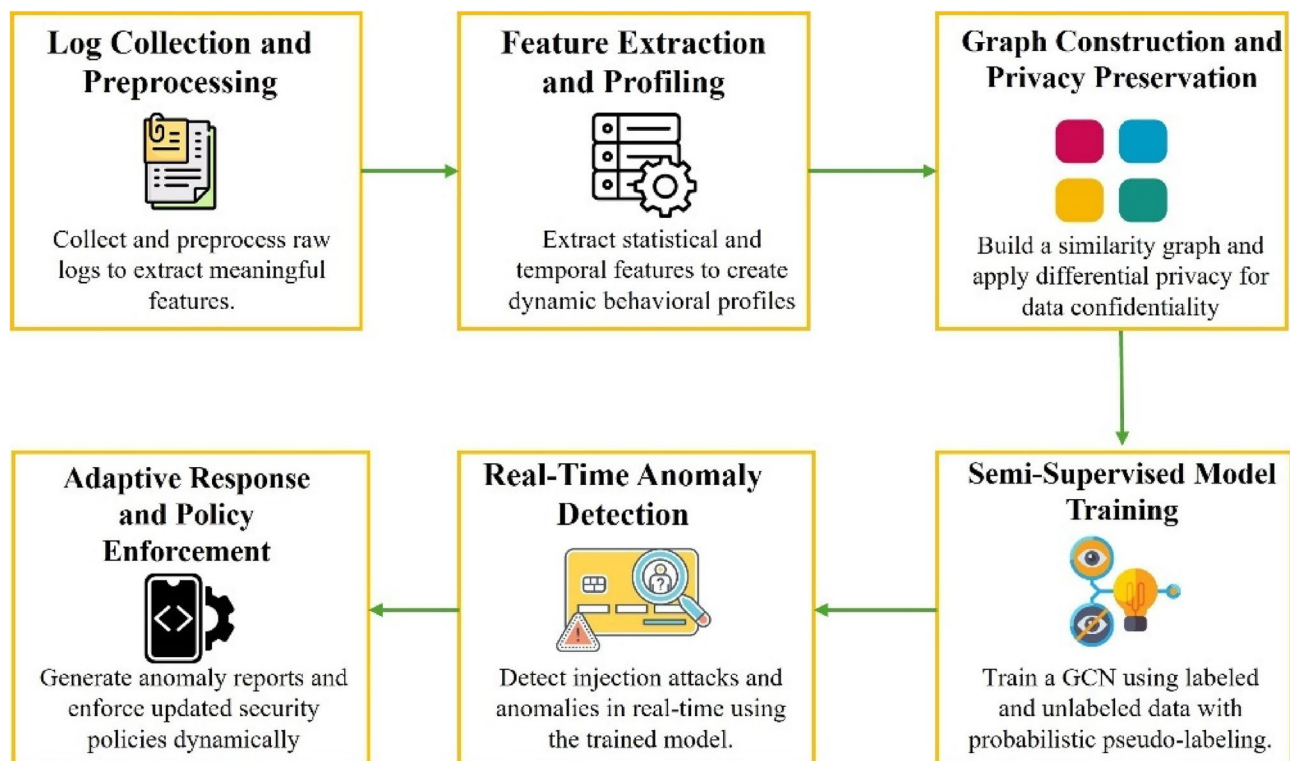


Fig. 1 Architecture of anomaly detection in cloud-based web applications

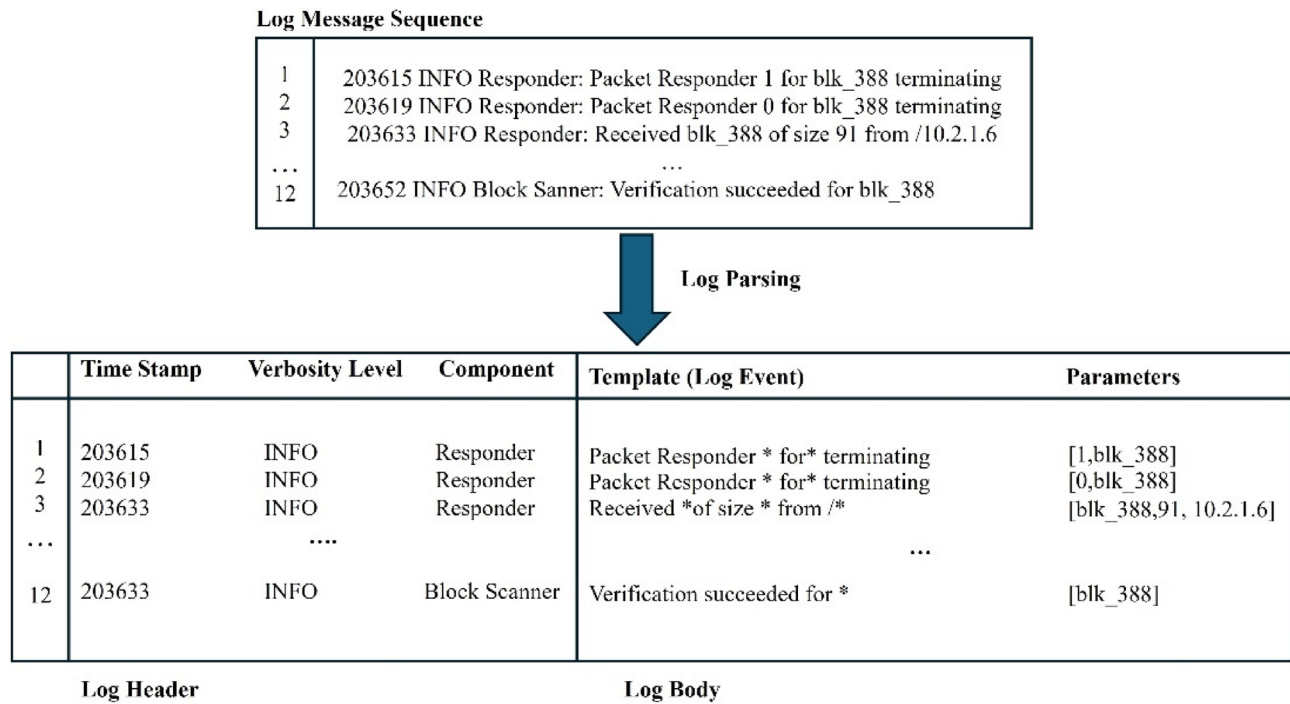


Fig. 2 An illustrative example of log terminology

Log body: The log body provides detailed descriptions of system activities using a combination of fixed templates and dynamic parameters.

- **Log Event (Template):** A predefined structure that describes the nature of the logged event. Placeholders, represented by asterisks (*), indicate variable content. These templates allow for the consistent representation of recurring events across different system components.
 - ✓ Packet Responder * for * terminating
 - ✓ Received * of size * from /
 - ✓ Verification succeeded for *
- **Log Parameters:** These are the dynamic values that fill in the placeholders within the log event templates. Parameters provide the contextual details needed for deeper analysis.
- **Log Parsing:** Log parsing is the process of converting raw, unstructured log messages into structured formats by identifying and separating the static templates from the dynamic parameters. This process is a critical component of the SSLA framework, as it enables the system to efficiently analyze logs for patterns, anomalies, and deviations from normal behavior.
- **Log Sequence:** A log sequence represents a series of related log events that originate from the same task, session, or data block. These sequences are essential

for tracking the life cycle of specific operations within the system.

In scenarios where explicit identifiers (e.g., block IDs or session IDs) are unavailable, the system employs sliding window techniques to group log events based on their timestamps. This technique aggregates logs occurring within defined time intervals, allowing for the identification of patterns and correlations even in the absence of direct identifiers. Sliding window techniques are particularly valuable for real-time anomaly detection in dynamic cloud environments.

Graph-Based anomaly detection using semi-supervised learning

In cloud-based web applications, injection attacks and anomalies often manifest as subtle disruptions in the system's behavior and communication patterns. Figure 3 illustrates the overall architecture of our method, which integrates knowledge graphs and attack graphs for robust anomaly monitoring.

- **Knowledge and Attack Graph Construction:** The system ingests both normal web application traffic and potential injection attacks, which are structured into two key graphs

Security Knowledge Graph: This graph models the relationships between different system entities (e.g., users, applications, servers) and their

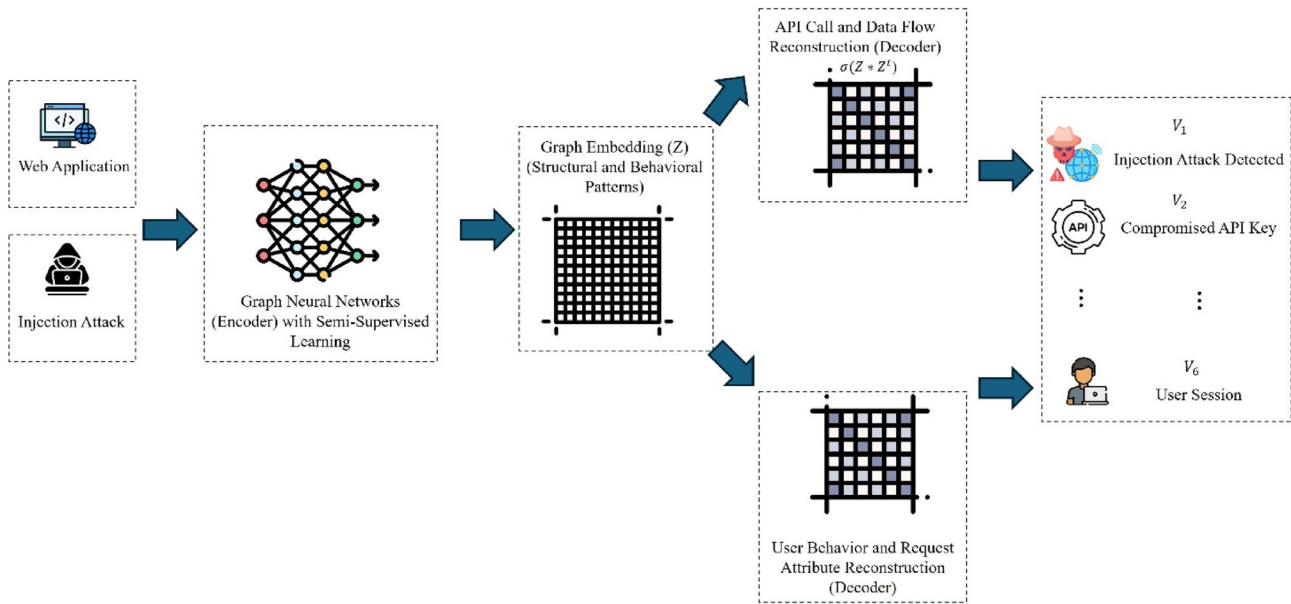


Fig. 3 Graph-based anomaly detection architecture using semi-supervised learning

interactions. Nodes represent entities, while edges encode relationships such as data flows, access requests, and API calls.

Attack Graph: This graph captures known attack patterns and injection behaviors. Nodes represent potential vulnerabilities or malicious actions (e.g., SQL injection attempts, XSS payloads), and edges illustrate the sequence or flow of an attack.

- **Graph Neural Network (Encoder):** The combined graphs are fed into a GNN, which learns meaningful embeddings for each node by aggregating information from their neighbors. The GNN consists of multiple hidden layers, each applying non-linear transformations (e.g., ReLU activations) to capture both structural and semantic patterns within the graph.

The embedding process outputs a matrix Z , where each row $\mathbf{z}_i$ represents the learned embedding of node v_i :

$$Z = [\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_n] \quad (1)$$

- **Graph Embedding and Reconstruction (Decoder):** The learned embeddings are then used to reconstruct both the structural and attribute information of the original graph through two decoders:

Structural Reconstruction (API Call and Data Flow): This decoder attempts to reconstruct the adjacency matrix of the graph by applying a similarity function $\sigma(ZZ^T)$, where Z^T is the transpose of the embedding matrix. A high

reconstruction error indicates potential anomalies in the relationships between entities.

Multi-Attribute Reconstruction (User Behavior and Request Attributes): This decoder focuses on reconstructing node attributes, such as user roles, request types, or data access levels. Discrepancies between the original and reconstructed attributes highlight abnormal behaviors or injection attempts.

- **Anomaly Evaluation:** The final step involves evaluating anomalies based on the reconstruction errors from both the structural and attribute decoders. Nodes with high reconstruction errors are flagged as potential anomalies. For example:
 - v_1 represents an injection attack detected through abnormal API call sequences.
 - v_2 corresponds to a compromised API key exhibiting unusual access patterns.
 - v_6 indicates a user session anomaly caused by unexpected request attributes.

The integration of knowledge graphs and attack graphs allows the system to detect both known attack patterns and novel anomalies, providing a comprehensive solution for real-time injection detection and anomaly monitoring in cloud-based web applications.

QoS considerations in anomaly detection

QoS is an Important for cloud-based web applications, ensuring reliable performance in real-time anomaly detection and injection monitoring. It helps maintain low latency, high availability, and efficient resource use,

critical for timely threat detection and system reliability. QoS parameters, such as latency, throughput, reliability, resource utilization, availability, and energy consumption, directly influence the performance and user experience of these applications. In anomaly detection systems, maintaining high QoS is critical to ensuring timely identification and mitigation of security threats without compromising the application's performance.

Latency and real-time detection

Anomaly detection mechanisms must operate with minimal latency to provide real-time insights and responses [48]. High detection latency can result in delayed threat identification, allowing malicious activities to persist and cause significant damage [49]. Techniques such as graph-based analysis and semi-supervised learning must be optimized to process large volumes of log data quickly. Parallel processing, distributed computing, and efficient data structures can be employed to reduce computational overhead and meet stringent latency requirements. The latency L is defined as:

$$L = t_{\text{detection}} - t_{\text{event}} \quad (2)$$

$t_{\text{detection}}$ indicate the timestamp when the anomaly is detected and t_{event} shows the timestamp when the event (e.g., log entry) occurred.

Throughput and data processing efficiency

Cloud-based environments often experience high volumes of data traffic, necessitating anomaly detection systems capable of handling large-scale data streams. Throughput, defined as the rate at which data is processed, must be maximized to ensure the system can scale effectively with increasing data loads [50]. The throughput T is defined as:

$$T = \frac{N}{\Delta t} \quad (3)$$

N shows the total number of log entries processed and Δt indicate the time interval over which the data was processed.

Resource Utilization (CPU, Memory) Metrics

Resource utilization (CPU, Memory) metrics

Resource utilization reflects the computational efficiency of the anomaly detection system, specifically in terms of CPU and memory usage [51]. The CPU utilization U_{CPU} is calculated as:

$$U_{\text{CPU}} = \frac{C_{\text{used}}}{C_{\text{total}}} \times 100\% \quad (4)$$

C_{used} demonstrate the amount of CPU cycles used by the anomaly detection system and C_{total} available CPU cycles.

Availability and fault tolerance

High availability is a cornerstone of cloud-based applications, requiring anomaly detection systems to function continuously without downtime [52]. Fault tolerance mechanisms, such as redundancy, failover strategies, and distributed architectures, ensure that the detection process remains operational even in the event of hardware or software failures [53]. The availability A is defined as:

$$A = \frac{\text{Uptime}}{\text{Uptime} + \text{Downtime}} \times 100\% \quad (5)$$

Uptime is the total time the system is operational, and Downtime is the total time the system is non-operational or under maintenance.

Energy consumption in Large-Scale detection scenarios

Energy consumption reflects the amount of power required by the anomaly detection system to process data and detect anomalies [54]. It is particularly important in large-scale or resource-constrained environments. The energy consumption E is calculated as:

$$E = P \times \Delta t \quad (6)$$

P indicate the average power consumption of the system (in watts). Δt indicate the time duration of operation (in hours or seconds).

Proposed method

Step 1: Log preprocessing

The first step in the proposed method involves preprocessing raw logs from cloud-based web applications to transform unstructured entries into a structured format suitable for real-time injection detection and anomaly monitoring. These logs, typically containing textual data, timestamps, and meta-data, are normalized, tokenized, and filtered to extract meaningful features while reducing noise for subsequent analysis.

Log representation

The log dataset is represented as $\mathcal{L} = [\mathcal{L}_1, \mathcal{L}_2, \dots, \mathcal{L}_N]$, where each log entry $\mathcal{L}_i$ is defined as:

$$\mathcal{L}_i = (\mathbf{x}_i, \mathbf{t}_i), \quad (7)$$

where $\mathbf{x}_i$ represents the unstructured textual content of the log entry (e.g., query strings, error messages, or transaction details), and $\mathbf{t}_i$ is the associated timestamp indicating when the log entry was generated.

Tokenization

The textual content $\mathbf{x}_i$ is transformed into a sequence of tokens $\mathbf{w}_i$ using a tokenizer $\mathcal{T}$, such that:

$$\mathcal{T} : \mathbf{x}_i \mapsto \mathbf{w}_i = [w_{i,2}, \dots, w_{i,m_i}], \quad (8)$$

where $\mathbf{w}_i$ denotes the sequence of tokens extracted from $\mathbf{x}_i$, $w_{i,j}$ represents the j -th token in the sequence, and m_i is the total number of tokens in $\mathbf{x}_i$. For instance, if $\mathbf{x}_i$ is the textual content “Login failed for user admin”, the tokenizer $\mathcal{T}$ might produce:

$$\mathbf{w}_i = [\text{Login}, \text{failed}, \text{for}, \text{user}, \text{admin}]. \quad (9)$$

Filtering

To reduce noise and eliminate irrelevant information, a filtering operator $\mathcal{F}$ is applied to the token sequence $\mathbf{w}_i$. The filtering process is represented as

$$\mathbf{w}_{i'} = \mathcal{F}(\mathbf{w}_i), \quad (10)$$

where $\mathbf{w}_{i'}$ contains only the meaningful tokens from $\mathbf{w}_i$. For the previous example, after filtering, the refined sequence becomes:

$$\mathbf{w}_{i'} = [\text{Login}, \text{failed}, \text{admin}]. \quad (11)$$

Step 2: Statistical feature extraction

To analyze statistical patterns inherent in the tokenized log sequences, a set of statistical features is derived for each processed sequence $\mathbf{w}_{i'}$. The extracted features include the *mean* (μ), *variance* (σ^2), *skewness* (γ_1), and *kurtosis* (γ_2), each providing distinct insights into the distribution of the token values.

The *mean* is defined as:

$$\mu(\mathbf{w}_{i'}) = \frac{1}{m_i} \sum_{j=1}^{m_i} w'_{i,j}, \quad (12)$$

where $\mu(\mathbf{w}_{i'})$ represents the average value of all tokens in the sequence $\mathbf{w}_{i'}$. In this context, m_i denotes the total number of tokens in the processed sequence, and $w_{i,j}'$ refers to the value of the j -th token in the sequence $\mathbf{w}_{i'}$. The index j starts from 1, corresponding to the first token. This formula calculates the arithmetic mean by summing up all token values in $\mathbf{w}_{i'}$ and dividing the result by the total number of tokens m_i . The mean (μ) is the average value of the token values in the sequence $\mathbf{w}_{i'}$, which serves as the central value around which the variance measures dispersion.

The variance, measuring the dispersion of token values around the mean, is given by:

$$\sigma^2(\mathbf{w}_{i'}) = \frac{1}{m_i} \sum_{j=1}^{m_i} (w_{i,j}' - \mu)^2, \quad (13)$$

where $\sigma^2(\mathbf{w}_{i'})$ quantifies how spread out the token values $w_{i,j}'$ are from their mean $\mu(\mathbf{w}_{i'})$. The mean (μ) is the average value of the token values in the sequence $\mathbf{w}_{i'}$, which serves as the central value around which the variance measures dispersion. The skewness, which measures the asymmetry of the token distribution, is defined as:

$$\gamma_1(\mathbf{w}_{i'}) = \frac{1}{m_i} \sum_{j=1}^{m_i} \left(\frac{w_{i,j}' - \mu}{\sigma} \right)^3, \quad (14)$$

where $\gamma_1(\mathbf{w}_{i'})$ captures whether the distribution of tokens $w_{i,j}'$ is skewed to the left ($\gamma_1 < 0$) or right ($\gamma_1 > 0$).

Finally, the kurtosis, indicating the tailedness of the token distribution, is defined as:

$$\gamma_2(\mathbf{w}_{i'}) = \frac{1}{m_i} \sum_{j=1}^{m_i} \left(\frac{w_{i,j}' - \mu}{\sigma} \right)^4, \quad (15)$$

where $\gamma_2(\mathbf{w}_{i'})$ determines whether the token distribution has heavier tails than a normal distribution ($\gamma_2 > 0$) or lighter tails ($\gamma_2 < 0$).

The statistical representation vector $\mathbf{f}_i$ for each log entry is then formed as:

$$\mathbf{f}_i = [\mu, \sigma^2, \gamma_1, \gamma_2]^T, \quad (16)$$

which compactly captures the key statistical properties of the token sequence $\mathbf{w}_{i'}$.

Step 3: Temporal feature embedding

To account for the sequential nature of logs, temporal features are incorporated by analyzing inter-arrival times. For each log entry, the inter-arrival time Δt_i is calculated as:

$$\Delta t_i = t_{i+1} - t_i, \quad \text{for } i < N, \quad (17)$$

where Δt_i represents the time difference between the timestamps t_i and t_{i+1} of consecutive log entries, and N is the total number of log entries in the dataset.

To capture temporal patterns over a sequence of log entries, a sliding window operator $\mathcal{W}$ of length ω is applied. This aggregates the inter-arrival times over a fixed window, generating a temporal feature vector $\mathbf{v}_i$ for each log entry i . The temporal feature vector is defined as:

$$\mathbf{v}_i = \mathcal{W}([\Delta t_{i-\omega+1}, \dots, \Delta t_i]). \quad (18)$$

In this equation, $\mathcal{W}$ is the sliding window operator, which aggregates inter-arrival times within a window of size ω . The parameter ω specifies the number of consecutive inter-arrival times used for aggregation, enabling the model to capture temporal patterns over a fixed interval. The inter-arrival time Δt_i represents the time difference

between two consecutive log entries, calculated using their respective timestamps t_i and t_{i+1} . The temporal feature vector $\mathbf{v}_i$ summarizes the temporal characteristics of log entries within the defined window.

Step 4: Semi-supervised label initialization with security considerations

To enhance robustness against adversarial labeling, a probabilistic approach is employed for pseudo-label assignment. This method incorporates a confidence threshold δ , ensuring that only high-confidence pseudo-labels are propagated. The updated pseudo-label assignment is defined as:

$$\hat{y}_j = \begin{cases} c, & \text{if } \Pr(\mathbf{f}_j|c) \geq \delta, \\ \text{reject}, & \text{otherwise,} \end{cases} \quad (19)$$

where c represents a candidate class, such as normal or anomalous behavior, and $\Pr(\mathbf{f}_j|c)$ is the posterior probability that the sample $\mathbf{f}_j$ belongs to class c . The threshold δ is a predefined value used to determine the minimum confidence required for assigning pseudo-labels. If $\Pr(\mathbf{f}_j|c)$ is below δ , the sample is rejected to prevent the propagation of low-confidence labels, thus enhancing the reliability of the method. The posterior probability $\Pr(\mathbf{f}_j|c)$ is calculated using Bayes' theorem:

$$\Pr(\mathbf{f}_j|c) = \frac{\Pr(c) \cdot \Pr(\mathbf{f}_j|c)}{\sum_{c'} \Pr(c') \cdot \Pr(\mathbf{f}_j|c')} \quad (20)$$

In this equation, $\Pr(c)$ represents the prior probability of class c , which reflects the proportion of samples in class c based on the labeled data. The term $\Pr(\mathbf{f}_j|c)$ is the likelihood, representing the probability of observing the feature vector $\mathbf{f}_j$ given class c , typically estimated from the training data. The denominator $\sum_{c'} \Pr(c') \cdot \Pr(\mathbf{f}_j|c')$ serves as a normalization term to ensure that the posterior probabilities for all possible classes sum to 1.

This probabilistic approach introduces several security advantages. By applying the confidence threshold δ , the method ensures that only pseudo-labels with sufficient certainty are propagated, thereby reducing the risk of propagating erroneous or noisy labels. Adversarial samples, which often result in low-confidence predictions, are effectively filtered out through the rejection mechanism. Additionally, the use of Bayesian posterior probabilities integrates prior knowledge with observed data, improving the overall reliability and robustness of the label assignment process.

Step 5: Graph construction and secure label propagation

To account for potential adversarial modifications, the label propagation process is enhanced by introducing a robust similarity metric. The similarity between two log entries is quantified as:

$$W_{ij} = \exp\left(-\frac{\|\mathbf{f}_i - \mathbf{f}_j\|}{2\sigma^2}\right) \quad (21)$$

This formulation ensures that closer feature vectors $\mathbf{f}_i$ and $\mathbf{f}_j$ are assigned higher similarity weights W_{ij} by utilizing the exponential function, which sharply decreases as the distance $\|\mathbf{f}_i - \mathbf{f}_j\|$ increases. The term $\|\mathbf{f}_i - \mathbf{f}_j\|$ measures the dissimilarity between the two vectors in the feature space, and smaller distances lead to larger similarity weights, emphasizing the relationship between closely related data points. The parameter σ^2 acts as a sensitivity controller, where smaller values make the function more selective by amplifying the decay rate, while larger values allow for a smoother weighting of moderately distant vectors. This design ensures that meaningful connections are prioritized in the graph, improving the accuracy and robustness of the label propagation process, even in scenarios involving noisy or adversarial modifications.

To secure the label propagation process, a differential privacy mechanism is employed. This mechanism ensures privacy while propagating labels across the similarity graph $\mathcal{G} = (V, E, W)$, where V represents nodes (log entries), E represents edges (connections between similar entries), and W encodes edge weights based on the similarity metric. The optimal label assignment $\mathbf{y}^*$ minimizes a privacy-preserving objective function:

$$\mathbf{y}^* = \underset{\mathbf{y}}{\operatorname{argmin}} \sum_{(i,j) \in E} W_{ij} \cdot (y_i - y_j)^2 + \epsilon \cdot \mathcal{R}(\mathbf{y}) \quad (22)$$

where $\mathcal{R}(\mathbf{y})$ is a regularization term that enforces differential privacy, and ϵ is the privacy budget, which controls the trade-off between privacy and utility. The term $(i, j) \in E$ indicates that the summation is performed over all edges E in the graph, where each edge connects node i and node j . The penalty term $W_{ij} \cdot (y_i - y_j)^2$ ensures smooth propagation of labels by penalizing differences between the labels y_i and y_j , with the penalty weighted by W_{ij} . Higher weights W_{ij} (representing greater similarity) encourage more similar labels for connected nodes, enforcing label smoothness across the graph while preserving relationships guided by the similarity metric.

Step 6: Dimensionality reduction

To reduce the computational complexity and noise associated with high-dimensional feature vectors, the log entries are projected into a lower-dimensional space using spectral embedding. This approach uses the graph structure defined in the previous step to preserve the relationships between log entries while reducing the dimensionality. The graph Laplacian L is computed as:

$$L = D - W \quad (23)$$

where D is the degree matrix, a diagonal matrix with elements defined as:

$$D_{ii} = \sum_j W_{ij}. \quad (24)$$

Here, W_{ij} represents the weight of the edge connecting nodes i and j , as defined in Step 5. The degree D_{ii} indicates the total edge weight associated with node i .

The spectral embedding is obtained by solving the eigenvalue problem for the graph Laplacian:

$$Lu = \lambda u, \quad (25)$$

where u is an eigenvector of L , and λ is the corresponding eigenvalue.

The top k eigenvectors $u_1, u_2, \dots, u_k$ corresponding to the smallest non-zero eigenvalues are selected to form the embedding. These eigenvectors capture the most important structural information of the graph.

The embedding z_i for each log entry i is then defined as:

$$\mathbf{z}_i = [u_{i,1}, u_{i,2}, \dots, u_{i,k}] \quad (26)$$

where $u_{i,j}$ is the j -th component of the i -th row in the selected eigenvectors. This embedding maps the log entries into a k -dimensional space while preserving their similarity structure from the graph.

In this equation, L represents the graph Laplacian, which encodes the relationship between nodes in the graph, and λ captures the spectral properties of the graph. The lower-dimensional representation $\mathbf{z}_i$ enables efficient processing and analysis while maintaining the most critical relationships between log entries.

Step 7: Semi-supervised model training

A semi-supervised learning model, such a GCN, is used to efficiently utilize both labeled and unlabeled input. The training process optimizes a combined loss function to balance supervised learning on labeled data and unsupervised learning on unlabeled data.

The total loss function is defined as:

$$\mathcal{L} = \mathcal{L}_{\text{sup}} + \lambda \mathcal{L}_{\text{unsup}}, \quad (27)$$

where $\mathcal{L}_{\text{sup}}$ is the supervised loss on labeled data, $\mathcal{L}_{\text{unsup}}$ is the unsupervised consistency loss on unlabeled data, and λ is a weighting parameter that controls the contribution of the unsupervised loss.

The supervised loss $\mathcal{L}_{\text{sup}}$ is calculated as the cross-entropy loss over the labeled dataset $\mathcal{D}_l$, which consists of pairs of feature vectors f_i and their corresponding labels y_i . The formula is:

$$\mathcal{L}_{\text{sup}} = \sum_{(f_i, y_i) \in \mathcal{D}_l} \sum_{c=1}^C y_{i,c} \log \hat{y}_{i,c} \quad (28)$$

where $\mathcal{D}_l$ represents the labeled dataset, containing pairs (f_i, y_i) . Each pair consists of a feature vector f_i , representing the features of log entry i , and its corresponding ground truth label y_i . The notation $(f_i, y_i) \in \mathcal{D}_l$ indicates that the summation is performed over all labeled data points i in the dataset $\mathcal{D}_l$. The term C denotes the total number of classes in the classification problem, while $c = 1$ to C refers to iterating over all possible classes for each labeled data point i . The ground truth label for log entry i in class c is represented as $y_{i,c}$, which is a binary value equal to 1 if the entry belongs to class c and 0 otherwise. The predicted probability that log entry i belongs to class c , as output by the model, is denoted by $\hat{y}_{i,c}$. The cross-entropy loss $\mathcal{L}_{\text{sup}}$ evaluates how well the model's predicted probabilities $\hat{y}_{i,c}$ align with the ground truth labels $y_{i,c}$. By maximizing the log of the predicted probability $\hat{y}_{i,c}$ for the correct class c , the model is encouraged to assign high probabilities to the correct classes for the labeled data points.

Step 8: Anomaly scoring

After training, the semi-supervised model assigns an anomaly score $S(f_i)$ to each log entry f_i . The anomaly score quantifies the likelihood of a log entry being anomalous, based on the model's predictions.

The anomaly score is defined as:

$$S(f_i) = 1 - \hat{y}_i(0), \quad (29)$$

where $\hat{y}_i(0)$ is the predicted probability of the log entry f_i being normal (class 0).

In this equation, $S(f_i)$ increases as the predicted probability of normality $\hat{y}_i(0)$ decreases. A high anomaly score indicates that the model considers the log entry likely to be anomalous, while a low score suggests normal behavior.

The model computes $\hat{y}_i(0)$ as part of its probabilistic output, which assigns probabilities to each possible class (e.g., normal and anomalous). The use of $1 - \hat{y}_i(0)$ ensures that the anomaly score is directly proportional to the model's confidence that the entry is anomalous.

For example, if $\hat{y}_i(0) = 0.95$, the anomaly score $S(f_i)$ would be $1 - 0.95 = 0.05$, indicating a low likelihood of the log entry being anomalous. Conversely, if $\hat{y}_i(0) = 0.2$, the anomaly score $S(f_i)$ would be $1 - 0.2 = 0.8$, indicating a high likelihood of anomaly. This scoring mechanism allows the system to rank log entries based on their anomaly scores, facilitating downstream tasks such as threshold-based anomaly detection or ranking for manual inspection.

Algorithm 2 delineates the processes of the proposed technique in detail.

Input: Raw log dataset $L = \{l_1, l_2, \dots, l_x\}$; confidence threshold δ ; privacy budget ε ; sliding window size ω

Output: Anomaly scores $S(f_i)$ for each log entry f_i

1. For each log entry $l_i \in L$:
2. Extract textual content x_i and timestamp t_i
3. Tokenize x_i into token sequence w_i using tokenizer T
4. Apply filtering operator F to obtain $w'_i = F(w_i)$
5. End for
6. For each filtered sequence w'_i :
7. Compute mean $\mu(w'_i)$, variance $\sigma^2(w'_i)$, skewness $\gamma_1(w'_i)$, kurtosis $\gamma_2(w'_i)$
8. Form statistical feature vector $f_i = [\mu, \sigma^2, \gamma_1, \gamma_2]^T$
9. End for
10. For $i = 1$ to $N - 1$:
11. Compute inter-arrival time $\Delta t_i = t_{e+1} - t_e$
12. End for
13. For $i = \omega$ to N :
14. Form sliding window $W = [\Delta t_{e-\omega+1}, \dots, \Delta t_e]$
15. Compute temporal feature vector $v_i = \text{aggregate}(W)$
16. End for
17. For each unlabeled entry f_r :
18. For each class $c \in [\text{normal}, \text{anomalous}]$:
19. Compute posterior probability $Pr(f_r | c)$
20. End for
21. End for
22. If $\max Pr(f_r | c) \geq \delta$ then
23. Assign pseudo-label $\hat{y}_r = \text{argmax}_k Pr(f_r | c)$
24. Else
25. Reject sample from pseudo-labeling
26. End if
27. End for
28. Initialize similarity graph $G = (V, E, W)$
29. For each pair (f_i, f_r) :
30. Compute similarity weight $W_{ir} = \exp(-||f_i - f_r||^2 / 2\sigma^2)$
31. If $W_{ir} > 0$ then
32. Add edge (i, j) with weight W_{ir} to E
33. End if
34. End for
35. Initialize labels y_i for labeled and pseudo-labeled nodes
36. While not converged do
37. For each node $i \in G$ do
38. Update y_i by minimizing: $\sum_{(i,j) \in E} W_{ir}(y_i - y_r)^2 + \varepsilon \cdot R(y)$
39. End for
40. End while
41. Compute degree matrix D , where $D_i = \sum_r W_{ir}$
42. Compute graph Laplacian $L = D - W$
43. Solve eigenvalue problem $Lu = \lambda u$
44. Select the k smallest non-zero eigenvectors $u_1, \dots, u_k$
45. For each node i do
46. Compute embedding $z_i = [u_{i1}, \dots, u_{ik}]$
47. End for
48. Initialize GCN model
49. For epoch = 1 to E :
50. Compute supervised loss L_{sup}
51. Compute unsupervised loss L_{unsup}
52. Total loss $L = L_{sup} + \lambda \cdot L_{unsup}$
53. Update model parameters via backpropagation
54. End for
55. For each log entry f_i :
56. Predict probability $\hat{y}_i(0)$ of being normal
57. Compute anomaly score $S(f_i) = 1 - \hat{y}_i(0)$
58. End for
59. Return all anomaly scores $S(f_i)$

Algorithm 2. SSLA– Semi-Supervised Log Analyzer

Table 3 Dataset summary

Dataset	Total Log Entries	Anomalies Detected
HDFS	11,175,629 log messages spanning 38.7 h, generated by 200 Amazon EC2 nodes running MapReduce tasks. The dataset consists of 575,062 log sequences from 29 log events.	16,838 (2.9%) sequences labeled as anomalies by domain experts.
BGL	4,747,963 log messages spanning 7 months, generated by the Blue Gene/L supercomputer with 128 K. Log sequences are extracted using fixed window partitioning with a window size of 200.	348,462 (7.34%) log messages marked as anomalies by domain experts.

Experimental results

The proposed SSLA framework was implemented and evaluated in a controlled experimental environment designed to simulate real-world cloud-based web applications. To ensure robust and scalable performance, the experiments were conducted using a private cloud infrastructure equipped with distributed computing resources. The experiments were carried out on a private cloud environment consisting of multiple virtual machines (VMs) hosted on a high-performance computing cluster. Each VM was provisioned with 16 vCPUs, 64 GB of RAM, and 1 TB of SSD storage, running on Intel Xeon Gold processors. The network infrastructure supported 10 Gbps Ethernet connections to ensure low-latency data transmission. The SSLA framework was implemented in Python, using machine learning libraries such as TensorFlow and PyTorch for the development of GCNs. Graph construction and similarity computations were handled using the NetworkX library, while data preprocessing and feature extraction were facilitated by Pandas. For parallel processing and distributed computation, Apache Spark was integrated into the environment, enabling efficient handling of large-scale log datasets. To simulate network behaviors and potential anomalies, the Network Simulator 3 (NS3) tool was used. This allowed the introduction of synthetic injection attacks and anomalies into the datasets, ensuring comprehensive evaluation under both real-world and simulated threat scenarios. Additionally, Docker containers were used to deploy microservices, replicating cloud-based web application environments where logs were generated and analyzed in real-time.

Dataset and evaluation metrics

We conduct experiments on two widely-used public datasets, namely the HDFS and BGL, to evaluate the

performance of the proposed anomaly detection framework. These datasets are chosen due to their real-world relevance, diversity in log structure, and the availability of labeled anomalies. The details of the datasets and the evaluation metrics are described below. Table 3 shows the summary of datasets and Table 4 comparison of HDFS and BGL Datasets.

Hadoop distributed file system (HDFS)

The HDFS dataset contains system logs collected from a Hadoop-based distributed file system running on a 200-node cluster over 24 days. These logs capture a variety of system operations, including block creation, deletion, replication, and transfer events. This dataset is widely used in anomaly detection research due to the complexity of distributed systems and the presence of real-world anomalies, such as system crashes and node failures.

Data collection period 24 days from a large-scale cluster.

Log structure Timestamp, log level (INFO, WARN, ERROR), component name, and message content.

Anomalies System failures including missing blocks, failed replications, and corrupted data transfers.

Challenge High volume and diversity in log patterns make it challenging to distinguish between normal operational logs and anomalies.

Log entry format example 2015-09-03 19:00:00, INFO, DataNode, Receiving block blk_12345 from/192.168.1.100

BlueGene/L (BGL)

The BGL dataset consists of system logs from the BlueGene/L supercomputer at Lawrence Livermore

Table 4 Comparison of HDFS and BGL Datasets

Feature	HDFS Dataset	BGL Dataset
Source	Hadoop Distributed File System logs	BlueGene/L Supercomputer logs
Log Complexity	Moderate, focused on file system operations	High, involving hardware and software-level events
Anomaly Types	Block failures, replication errors, data corruption	Hardware failures, voltage issues, node crashes
Temporal Correlation	Low to moderate	High
Typical Use Case	Cloud computing systems, distributed file systems	High-performance computing, supercomputers

National Laboratory, one of the most powerful supercomputers in the world during its operation. The logs capture a wide range of system events, from hardware-level signals to high-level software operations. The dataset is rich in temporal correlations, which makes it particularly suitable for testing the robustness of anomaly detection algorithms.

Data collection period Several months of operational logs.

Log structure Timestamp, event type (RAS “Reliability, Availability, Serviceability”), severity level (*INFO*, *WARN*, *ERROR*), hardware location, and detailed error message.

Anomalies Hardware failures, node crashes, voltage fluctuations, and temperature out-of-range errors.

Challenge High-dimensional data with intricate temporal dependencies, making anomaly patterns less obvious.

Log entry format example 2005-07-04 12:23:45, RAS, ERROR, R05-M0-NC-J05-U11-C4, Voltage out of range

Comparison algorithms

This section provides a detailed overview of four state-of-the-art algorithms for anomaly detection in cloud-based web applications. These algorithms are compared with the proposed SSLA framework to assess performance, scalability, and robustness in detecting injection attacks and anomalous behaviors. Most existing anomaly detection frameworks in cloud environments rely heavily on either fully supervised or unsupervised learning approaches, which limits their effectiveness in real-world scenarios where labeled data is scarce and log data is often semi-structured. In contrast, SSLA introduces a novel end-to-end semi-supervised framework that effectively integrates both labeled and unlabeled log entries using graph-based modeling and label propagation techniques.

DeepLog

Deeplog is a deep learning-based anomaly detection framework specifically designed for system log analysis. It uses Long Short-Term Memory (LSTM) networks to model normal sequences of log events, identifying anomalies based on deviations from these learned patterns. DeepLog preprocesses log entries to extract event IDs and uses LSTM to predict the next log event in a sequence. Anomalies are detected when the observed event deviates significantly from the predicted event [55].

LogAnomaly

LogAnomaly integrates sequential modeling and quantitative feature extraction to enhance anomaly detection in system logs. It uses a combination of LSTM networks and statistical feature analysis for robust detection. LogAnomaly extracts both sequential patterns and statistical features (e.g., frequency, time gaps) from logs. The LSTM captures temporal dependencies, while statistical features provide contextual insights. The outputs are combined for final anomaly detection [56].

Graph convolutional networks (GCNs)

Graph-based Anomaly Detection (GAD) methods, such as those using GCNs, offer greater flexibility by capturing structural relationships between log entries. However, these models often assume a fully connected graph and typically overlook considerations of privacy and real-time performance. The proposed SSLA framework addresses these limitations by introducing a privacy-aware label propagation scheme, which incorporates differential privacy directly into the graph learning process. This ensures the confidentiality of sensitive data without compromising detection performance [57].

Isolation forest (iForest)

While efficient for high-dimensional anomaly detection, assumes that anomalies are rare and distinct, which may not hold in evolving cloud environments. Moreover, iForest has a deficiency in temporal awareness and fails to use contextual linkages among log entries. An important difference is SSLA’s highlighting on QoS measures, which has often been neglected in previous research. Although the majority of current models prioritize detection accuracy, SSLA is explicitly designed to sustain minimal latency, energy consumption, and CPU utilization—essential criteria in real-time cloud settings [56].

These algorithms provide diverse approaches to anomaly detection, enabling a comprehensive comparison with the proposed SSLA framework. Each method offers unique strengths and limitations, contributing to a holistic evaluation of detection capabilities in cloud-based web applications.

Performance evaluation of anomaly detection methods

The performance metrics used for evaluation include Precision (Prec), Recall (Rec), and F1-score (F1). These metrics are defined as follows:

Table 5 Results on the Stable HDFS Dataset

Method	Prec	Rec	F1
DeepLog	0.95	0.93	0.94
LogAnomaly	0.96	0.94	0.95
GCN (GAD)	0.97	0.95	0.96
iForest	0.90	0.85	0.87
SSLA (Proposed)	0.98	0.97	0.975

Table 6 Results on Unstable BGL Dataset

Method	Train=0.8			Train=0.5		
	P	R	F1	P	R	F1
DeepLog	0.92	0.90	0.91	0.88	0.85	0.86
LogAnomaly	0.93	0.91	0.92	0.89	0.86	0.87
GCN (GAD)	0.94	0.92	0.93	0.90	0.87	0.88
iForest	0.88	0.84	0.86	0.82	0.78	0.80
SSLA	0.96	0.94	0.95	0.92	0.90	0.91

$$\text{Precision} = \frac{TP}{TP + FP} \quad (30)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (31)$$

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (32)$$

where TP represents True Positives, FP represents False Positives, and FN represents False Negatives. High precision indicates a low false positive rate, while high recall reflects the method's ability to detect most anomalies. The F1-score provides a balance between precision and recall.

Results on the Stable HDFS Dataset

Table 5 presents the performance of various anomaly detection methods on the stable HDFS dataset. The proposed SSLA method achieves the highest precision, recall, and F1-score, demonstrating its superior capability in identifying anomalies in a stable environment.

Results on Unstable BGL Dataset

Table 6 shows the performance comparison on the unstable BGL dataset with two different training ratios, 0.8 and 0.5. The proposed SSLA consistently achieves higher precision, recall, and F1-scores compared to other methods, indicating its robustness and ability to handle dynamic, noisy data environments.

Results Comparison Between Variants of LogOnline

Table 7 compares the results of different variants of the LogOnline method. The proposed SSLA achieves the highest precision, recall, and F1-score, outperforming even the best-performing variant, LogOnline-Var3. This indicates that SSLA offers a more effective approach to anomaly detection.

Table 7 Results Comparison Between Variants of LogOnline

Variant	Prec	Rec	F1
LogOnline-Var1	0.94	0.92	0.93
LogOnline-Var2	0.95	0.93	0.94
LogOnline-Var3	0.96	0.94	0.95
SSLA (Proposed)	0.97	0.96	0.965

QoS evaluation on HDFS dataset

Figure 4 presents the availability comparison of five anomaly detection methods DeepLog, LogAnomaly, Graph GCNs, iForest, and the proposed SSLA on the HDFS dataset as the number of processed log entries increases. The proposed SSLA consistently achieves the highest availability, starting at 99.9% and maintaining 99.7% even when processing 1.1×10^7 log entries, demonstrating its robustness and scalability in large-scale environments. GCNs and LogAnomaly also perform well initially, with availability starting at 99.85% and 99.8%, respectively, but both experience more noticeable declines, dropping to 99.45% and 99.4% as data volume increases. DeepLog shows moderate availability, decreasing from 99.7 to 99.3%, reflecting its limitations in handling larger datasets due to its sequential LSTM-based architecture. iForest exhibits the lowest availability, declining from 99.5 to 99.1%, highlighting its inefficiency in managing complex anomalies in high-dimensional data. Overall, SSLA outperforms the other methods, maintaining superior availability across all scales of data processing.

In the Fig. 5 proposed SSLA achieves the lowest latency, starting at 110 ms and increasing gradually to 150 ms when processing log entries, highlighting its real-time processing capabilities and efficiency in large-scale environments. In contrast, iForest exhibits the highest latency, rising from 170 ms to 210 ms, indicating its limitations in handling complex log data in time-sensitive applications. DeepLog and LogAnomaly show moderate latency, starting at 150 ms and 140 ms, respectively, and increasing to 190 ms and 180 ms as data volume grows, reflecting the computational overhead of sequential processing models. GCNs maintain relatively lower latency compared to traditional models, ranging from 130 ms to 170 ms, but still fall short of SSLA's performance. Overall, SSLA outperforms all other methods by maintaining the lowest latency across varying data volumes, making it highly suitable for real-time anomaly detection in cloud-based applications.

Figure 6 illustrates the CPU utilization comparison of five anomaly detection methods—DeepLog, LogAnomaly, GCNs, iForest, and the proposed SSLA—on the HDFS dataset as the number of processed log entries increases. The proposed SSLA demonstrates the most

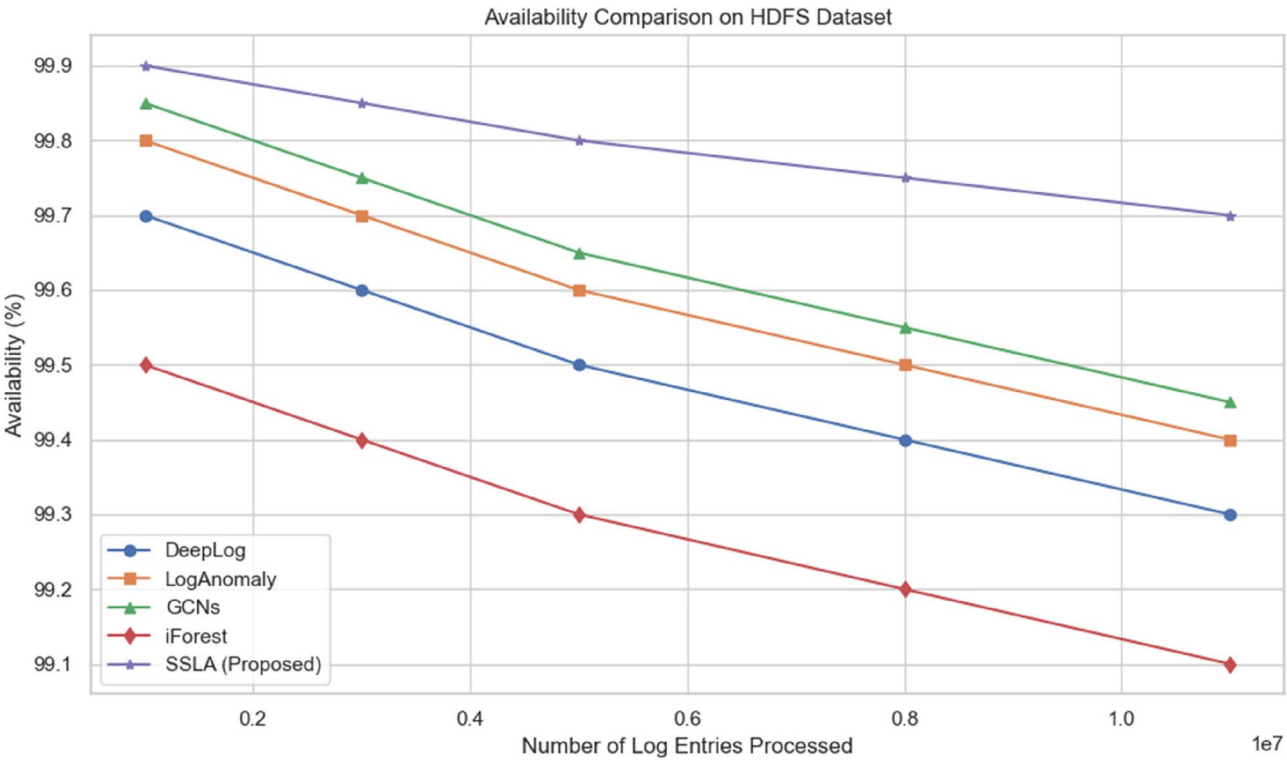


Fig. 4 Availability comparison on HDFS dataset

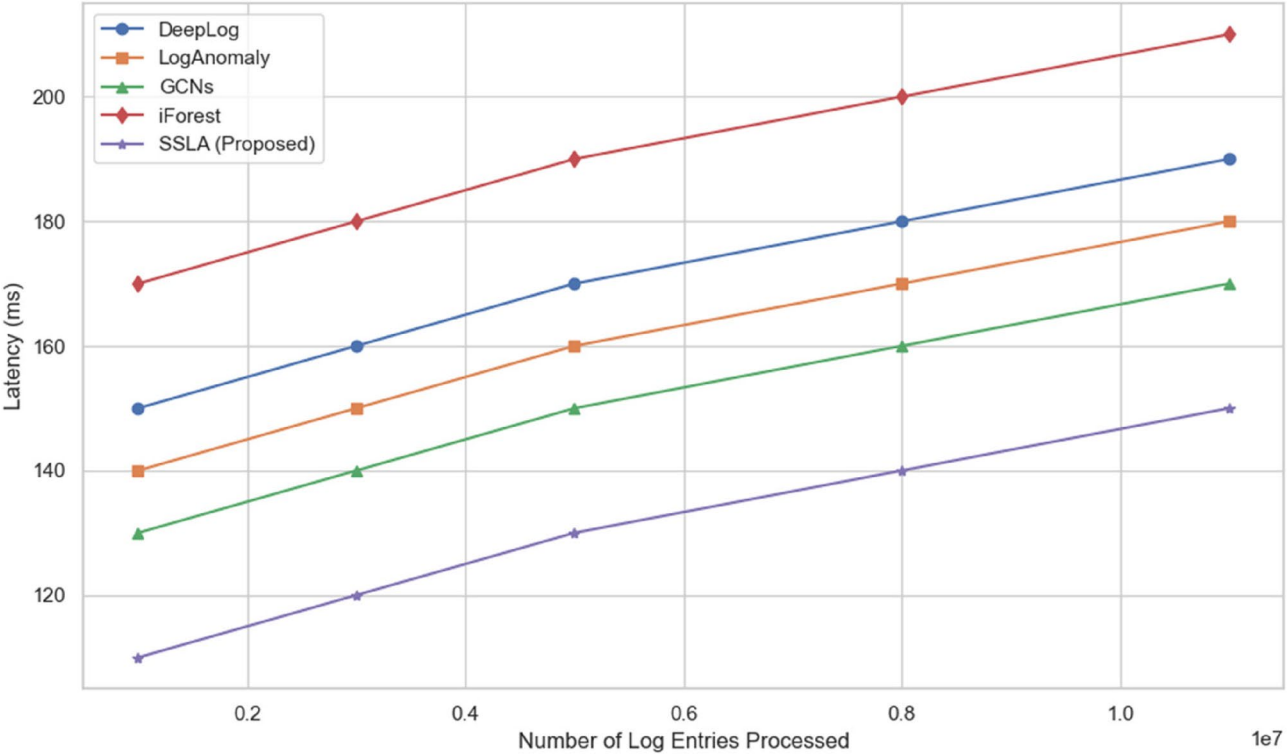


Fig. 5 Detection latency comparison on HDFS dataset

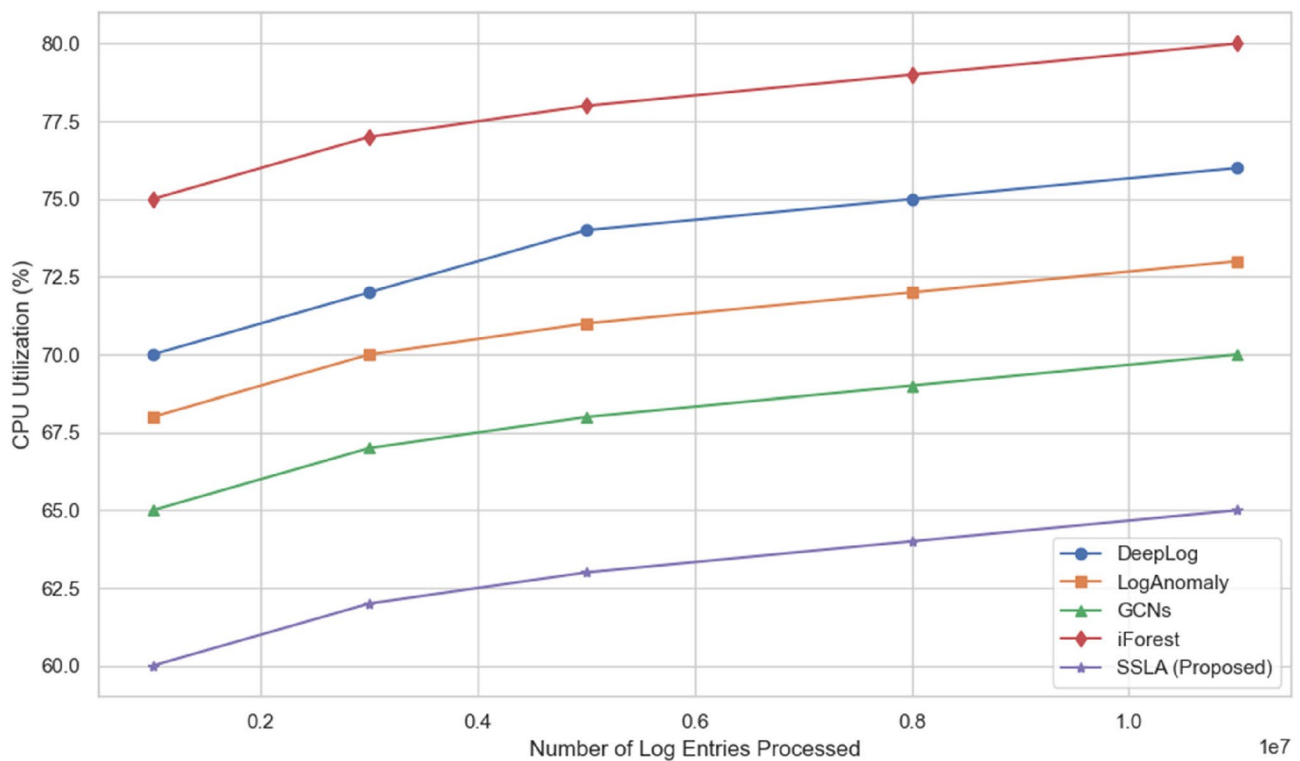


Fig. 6 CPU utilization comparison on HDFS dataset

efficient CPU utilization, starting at 60% and rising modestly to 65% when processing 1.1×10^7 log entries, indicating its computational efficiency and scalability in handling large datasets. In contrast, iForest exhibits the highest CPU usage, increasing from 75 to 80%, reflecting its inefficiency in managing complex log data. DeepLog and LogAnomaly show moderate CPU utilization, starting at 70% and 68%, respectively, and increasing to 76% and 73%, suggesting higher computational overhead due to their sequential processing models. GCNs maintain relatively lower CPU usage among traditional models, ranging from 65 to 70%, but still exceed SSLA in resource consumption. Overall, SSLA outperforms the other methods by maintaining the lowest CPU utilization across all data volumes, highlighting its suitability for resource-constrained, real-time cloud-based applications.

Figure 7 shows the proposed SSLA consistently demonstrates the lowest energy consumption, starting at 220 watts and rising to 380 watts when processing log entries, highlighting its energy efficiency and suitability for large-scale applications. In contrast, iForest exhibits the highest energy consumption, increasing from 270 watts to 470 watts, reflecting its inefficiency in handling high-dimensional log data. DeepLog and LogAnomaly show moderate energy usage, starting at 250 watts and 240 watts, respectively, and increasing to 450 watts and 440 watts, indicating higher computational demands due to

their sequential processing and feature extraction methods. GCNs maintain slightly lower energy consumption than DeepLog and LogAnomaly, ranging from 230 watts to 430 watts, but still consume more energy than SSLA. Overall, SSLA outperforms the other methods by maintaining the lowest energy consumption across all data volumes, demonstrating its effectiveness in energy-constrained, real-time cloud-based environments.

Figure 8 illustrates the throughput comparison of five anomaly detection methods—DeepLog, LogAnomaly, GCNs, iForest, and the proposed SSLA—on the HDFS dataset as the number of processed log entries increases. The proposed SSLA consistently achieves the highest throughput, starting at 900 log entries per second and decreasing slightly to 860 log entries per second when processing 1.1×10^7 log entries, demonstrating its superior efficiency in handling large-scale data. GCNs and LogAnomaly follow, with initial throughputs of 870 and 860 log entries per second, respectively, gradually declining to 830 and 820 log entries per second, reflecting their moderate scalability in high-volume environments. DeepLog shows slightly lower throughput, starting at 850 log entries per second and decreasing to 810 log entries per second, indicating its reduced efficiency due to sequential processing overhead. iForest consistently exhibits the lowest throughput, declining from 800 to 760 log entries per second, highlighting its limitations in processing large datasets efficiently. Overall, SSLA

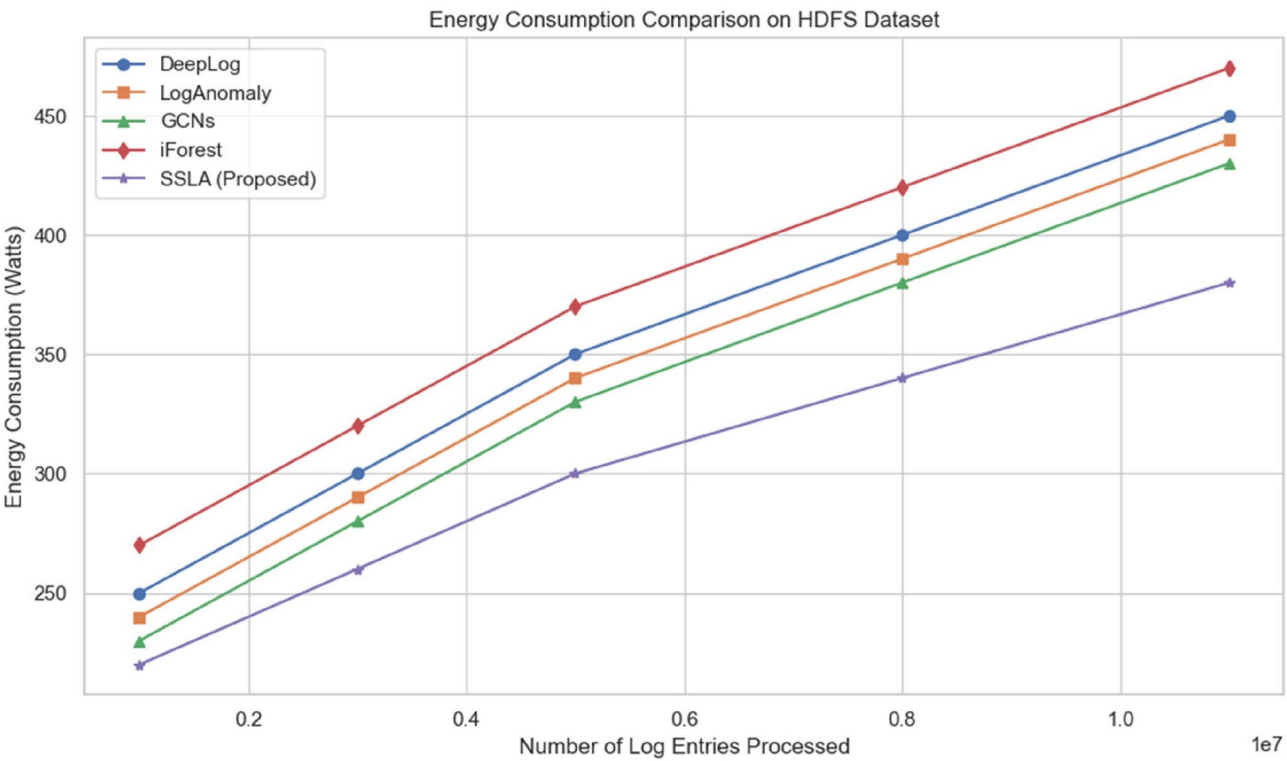


Fig. 7 Energy consumption comparison on HDFS dataset

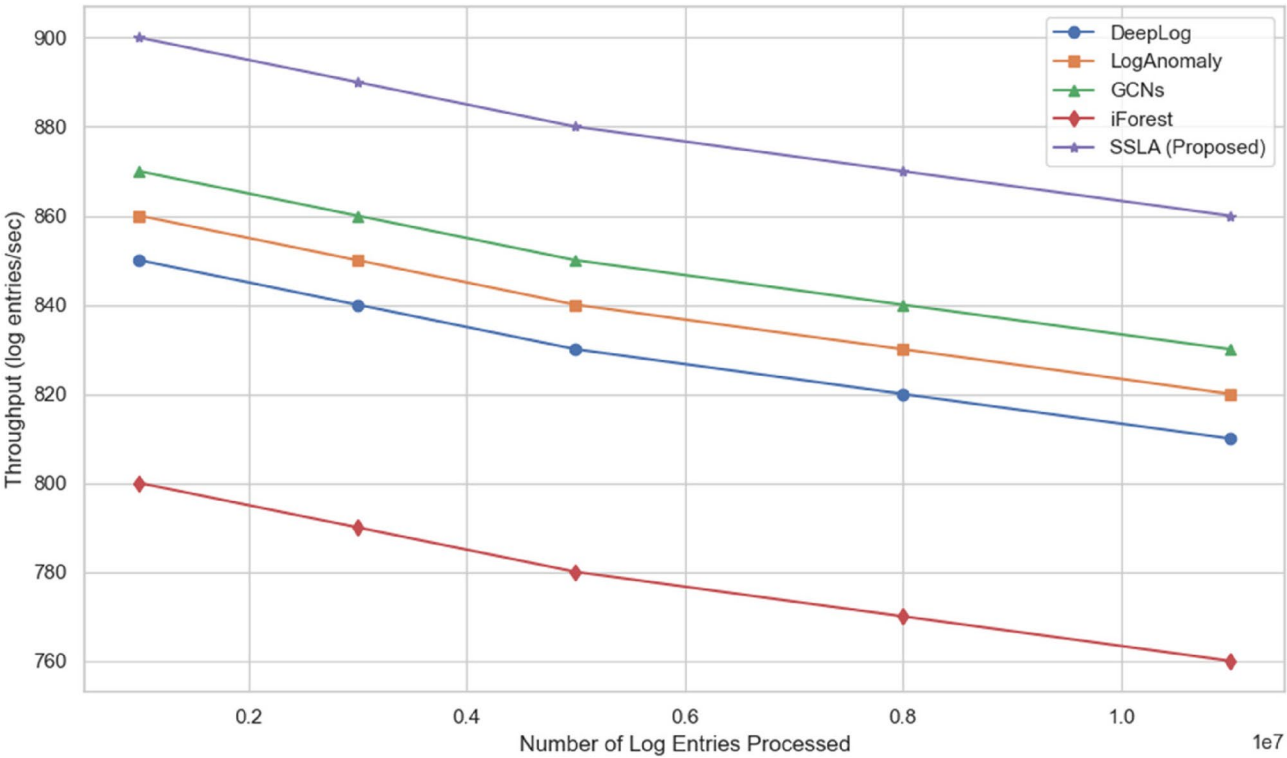


Fig. 8 Throughput comparison on HDFS dataset

outperforms the other methods by maintaining the highest throughput across all data volumes, making it highly suitable for real-time anomaly detection in cloud-based applications.

QoS evaluation on BGL dataset

Figure 9 presents the availability comparison of five anomaly detection methods—DeepLog, LogAnomaly, GCNs, iForest, and the proposed SSLA—on the BGL dataset as the number of processed log entries increases. The proposed SSLA consistently achieves the highest availability, starting at 99.8% and maintaining 99.6% even when processing log entries, demonstrating its robustness and scalability in large-scale environments. GCNs and LogAnomaly follow with slightly lower availability, beginning at 99.65% and 99.6%, respectively, and gradually decreasing to 99.25% and 99.2% as the data volume increases. DeepLog shows moderate availability, decreasing from 99.5 to 99.1%, reflecting its limitations in handling large datasets due to its sequential processing architecture. iForest exhibits the lowest availability, starting at 99.3% and dropping to 98.9%, highlighting its inefficiency in maintaining stable performance under high data loads. Overall, SSLA outperforms the other methods by maintaining superior availability across all scales of data processing, making it highly suitable for real-time anomaly detection in high-performance computing environments like BGL.

Figure 10 illustrates the CPU utilization comparison of five anomaly detection methods—DeepLog, LogAnomaly, GCNs, iForest, and the proposed SSLA—on the BGL dataset as the number of processed log entries increases. The proposed SSLA demonstrates the most efficient CPU utilization, starting at 65% and rising moderately to 72% when processing log entries, showcasing its computational efficiency and scalability for large-scale anomaly detection tasks. In contrast, iForest exhibits the highest CPU utilization, increasing from 80 to 87%, reflecting its inefficiency in handling complex log data. DeepLog and LogAnomaly show moderate CPU usage, starting at 75% and 73%, respectively, and increasing to 82% and 80%, indicating higher computational overhead due to their sequential processing frameworks. GCNs maintain slightly lower CPU usage among traditional models, ranging from 70 to 77%, but still exceed SSLA in resource consumption. Overall, SSLA outperforms the other methods by maintaining the lowest CPU utilization across all data volumes, making it highly suitable for resource-constrained, real-time anomaly detection in high-performance computing environments like BGL.

Figure 11 shows the energy consumption comparison of five anomaly detection methods—DeepLog, LogAnomaly, GCNs, iForest, and the proposed SSLA—on the BGL dataset as the number of processed log entries increases. The proposed SSLA consistently demonstrates the lowest energy consumption, starting at 260 watts and rising to 500 watts when processing log entries,

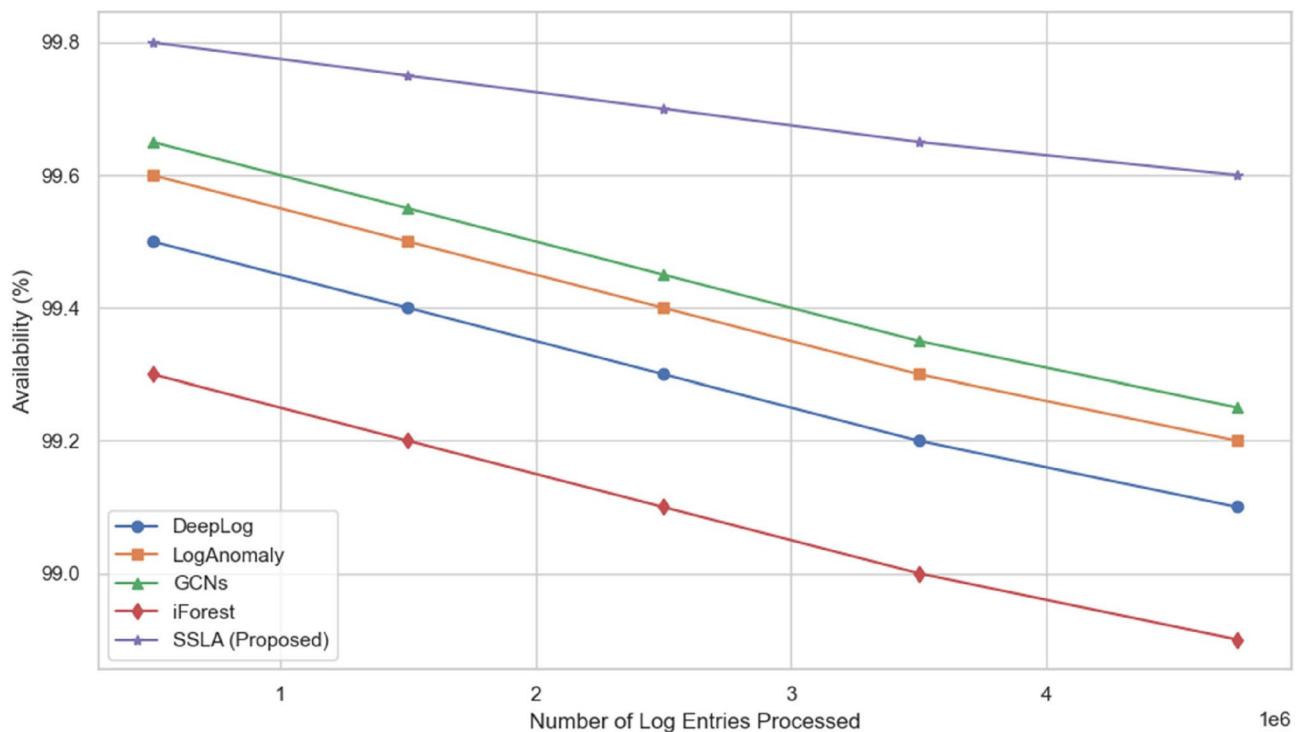


Fig. 9 Availability comparison on BGL dataset

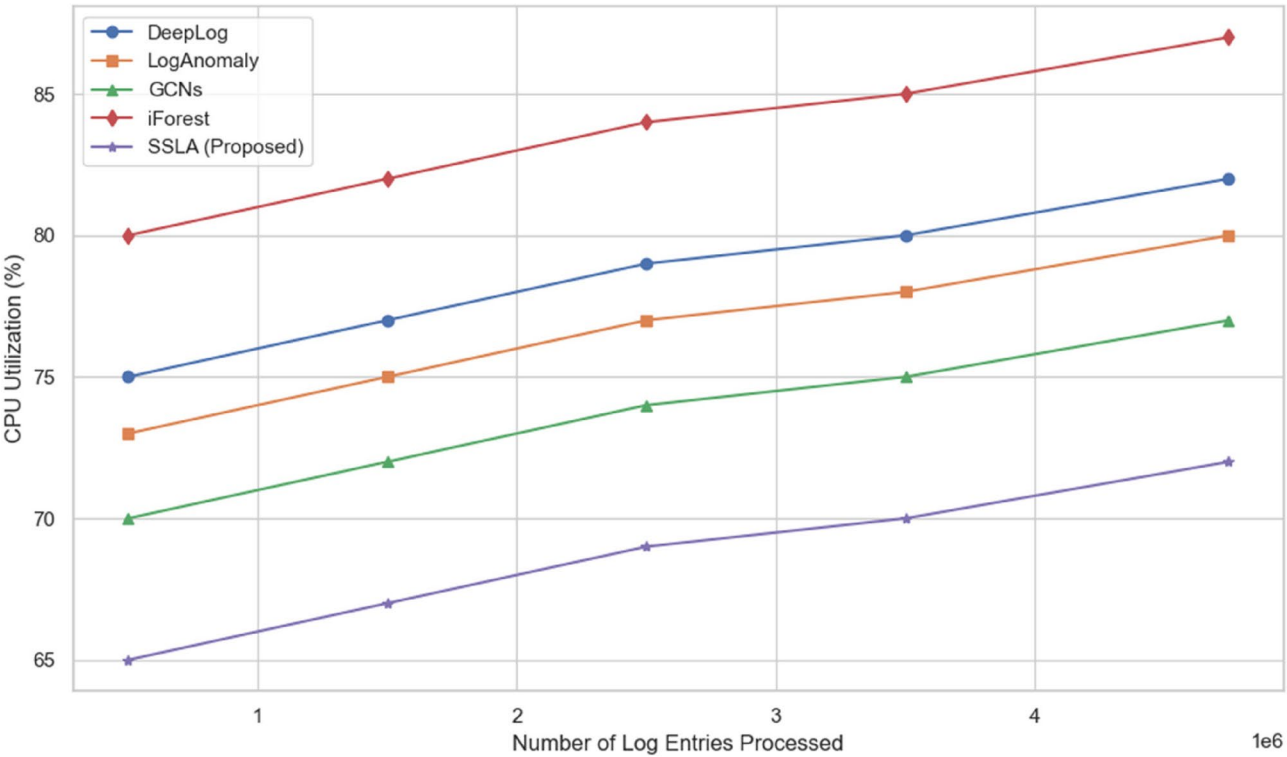


Fig. 10 CPU utilization comparison on BGL dataset

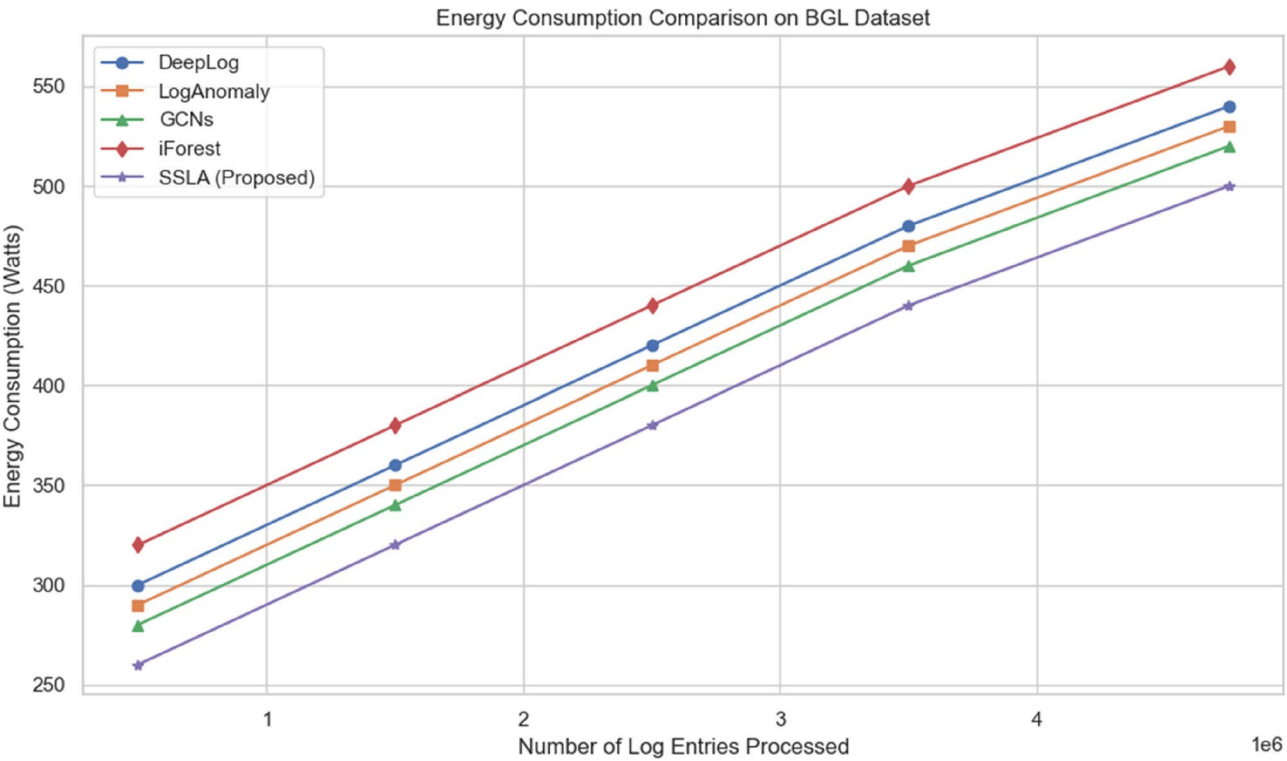


Fig. 11 Energy consumption comparison on BGL dataset

highlighting its energy efficiency and suitability for large-scale anomaly detection tasks. In contrast, iForest exhibits the highest energy consumption, increasing from 320 watts to 560 watts, reflecting its inefficiency in managing high-dimensional log data. DeepLog and LogAnomaly show moderate energy usage, starting at 300 watts and 290 watts, respectively, and increasing to 540 watts and 530 watts, indicating higher computational demands due to their sequential processing and feature extraction mechanisms. GCNs maintain slightly lower energy consumption than DeepLog and LogAnomaly, ranging from 280 watts to 520 watts, but still consume more energy than SSLA. Overall, SSLA outperforms the other methods by maintaining the lowest energy consumption across all data volumes, making it highly suitable for energy-constrained, real-time anomaly detection in high-performance computing environments like BGL.

Figure 12 illustrates the detection latency comparison of five anomaly detection methods—DeepLog, LogAnomaly, GCNs, iForest, and the proposed SSLA—on the BGL dataset as the number of processed log entries increases. The proposed SSLA consistently achieves the lowest detection latency, starting at 140 ms and rising to 180 ms when processing log entries, underscoring its efficiency in real-time anomaly detection scenarios. In contrast, iForest exhibits the highest latency, increasing from 200 ms to 240 ms, reflecting its inefficiency in handling high-volume data in time-sensitive

applications. DeepLog and LogAnomaly show moderate latency, starting at 180 ms and 170 ms, respectively, and increasing to 220 ms and 210 ms, highlighting the computational overhead associated with their sequential processing techniques. GCNs maintain slightly lower latency among traditional models, ranging from 160 ms to 200 ms, but still fall short of SSLA's performance. Overall, SSLA outperforms the other methods by maintaining the lowest latency across all data volumes, making it highly suitable for real-time anomaly detection in high-performance computing environments like BGL.

Figure 13 presents the throughput comparison of five anomaly detection methods—DeepLog, LogAnomaly, GCNs, iForest, and the proposed SSLA—on the BGL dataset as the number of processed log entries increases. The proposed SSLA consistently achieves the highest throughput, starting at 800 log entries per second and decreasing slightly to 760 log entries per second when processing 4.8×10^6 log entries, highlighting its efficiency in maintaining high data processing rates under increasing workloads. In contrast, iForest exhibits the lowest throughput, starting at 700 log entries per second and declining to 660 log entries per second, reflecting its inefficiency in handling large-scale data in high-performance environments. GCNs and LogAnomaly demonstrate moderate throughput, beginning at 770 and 760 log entries per second, respectively, and reducing to 730 and 720 log entries per second as the data volume increases.

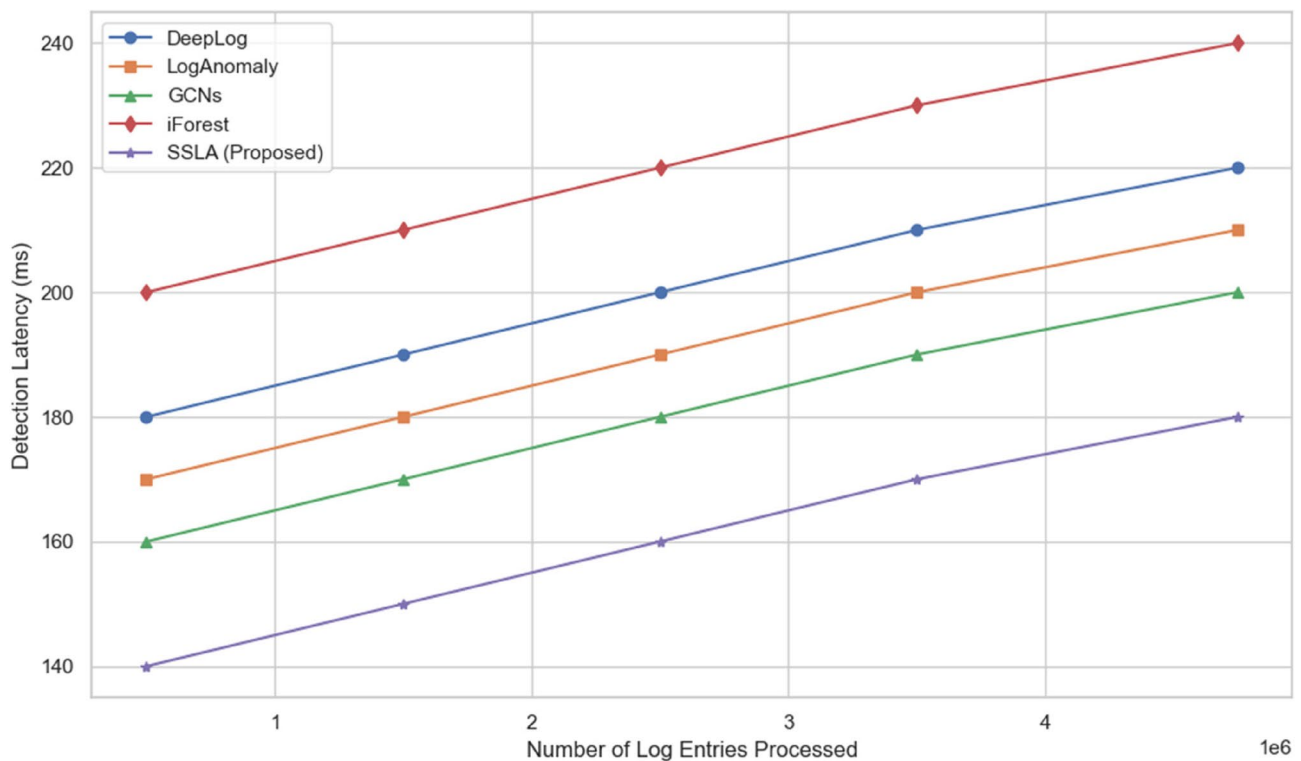


Fig. 12 Detection latency comparison on BGL dataset

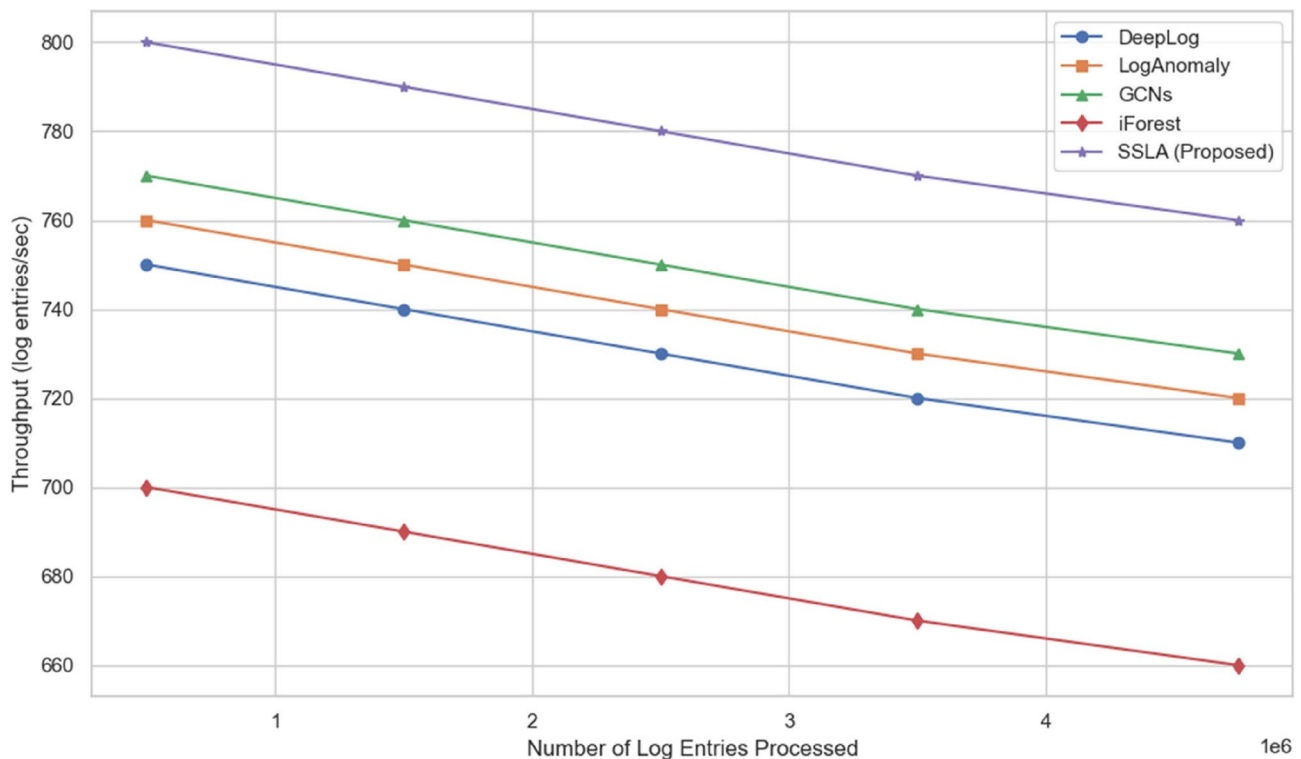


Fig. 13 Throughput comparison on BGL dataset

DeepLog follows a similar trend, starting at 750 log entries per second and dropping to 710 log entries per second, indicating the impact of sequential processing on throughput. Overall, SSLA outperforms the other methods by maintaining the highest throughput across all data volumes, making it highly suitable for real-time, large-scale anomaly detection in high-performance computing environments like BGL.

Results and discussion

A comprehensive examination of the proposed SSLA vs. established anomaly detection methodologies—DeepLog, LogAnomaly, GCNs, and iForest—was performed across several performance metrics: availability, CPU utilization, energy consumption, detection delay, and throughput. The experimental evaluations were conducted on the HDFS and BGL datasets.

Availability

The availability metric demonstrates that SSLA consistently achieves higher system uptime across both datasets. Specifically, on the HDFS dataset, SSLA maintains availability above 99.7% even as log volume scales, outperforming GCNs and DeepLog, which experience more pronounced availability degradation. This robustness stems from SSLA's dynamic graph construction and

differential privacy mechanisms, which effectively mitigate false positives and ensure the system remains operational without unwarranted interruptions. Methods like iForest, which rely heavily on isolation heuristics, suffer from overfitting to normal patterns, thereby increasing false alarms and reducing availability. The graph-based propagation in SSLA enhances anomaly localization without compromising legitimate system processes, contributing to its superior availability.

CPU utilization

Proposed method has significant advantage in CPU efficiency, maintaining the lowest utilization rates across both datasets. This is attributed to the integration of spectral embedding techniques that reduce the dimensionality of log data while preserving important structural relationships. The semi-supervised learning architecture used by SSLA enhances computing efficiency by using both labeled and unlabeled data, hence reducing the need for extensive model retraining. Conversely, DeepLog and iForest demonstrate elevated CPU use owing to their dependence on sequential data traversal and recursive partitioning, respectively. The inherent computational complexity of these models becomes a bottleneck as log volumes increase, whereas SSLA's graph-based approach facilitates parallel processing and efficient resource allocation.

Energy consumption

The proposed method consistently consumes less energy compared to its counterparts, which can be directly linked to its efficient feature extraction and graph construction methodologies. SSLA eliminates energy-intensive procedures often seen in models such as iForest and GCNs by minimizing unnecessary calculations using probabilistic pseudo-labeling and secure label transmission. The energy overhead in these models arises from their exhaustive search mechanisms and iterative training processes, which are inherently less scalable. SSLA's ability to maintain low energy consumption while processing large-scale log data demonstrates its suitability for deployment in resource-constrained cloud environments.

Detection latency

Detection latency is an important parameter for real-time anomaly detection systems, and SSLA has a distinct superiority in this area. The model attains the minimal latency for both HDFS and BGL datasets, attributable to its efficient semi-supervised learning framework and strong similarity metrics. SSLA's are specifically tailored to propagate labels swiftly and accurately, even in the presence of adversarial inputs. This stands in contrast to iForest and DeepLog, which exhibit higher latencies due to their reliance on sequential anomaly scoring and deep recurrent architectures, respectively. The fast convergence of SSLA's learning process, aided by spectral embedding and dimensionality reduction, allows for timely anomaly identification without compromising accuracy.

Throughput

SSLA's has a superior throughput performance, maintaining the highest log processing rates across varying data volumes. The model's scalability is driven by its ability to construct dynamic similarity graphs that adapt to evolving data patterns, allowing for efficient parallel processing. Methods such as DeepLog and LogAnomaly, while effective in static environments, struggle to maintain high throughput in dynamic, large-scale settings due to their less efficient data representation and processing frameworks. The reduction in throughput observed in iForest and GCNs is indicative of their higher computational complexity and inability to handle increasing data volumes efficiently. SSLA's integration of dimensionality reduction techniques and secure label propagation ensures sustained throughput, making it highly effective for real-time applications.

Architectural insights and performance rationalization

The superior performance of SSLA across all evaluated metrics can be attributed to its hybrid architectural

design, which synergizes semi-supervised learning with privacy-preserving graph techniques. Spectral embedding for dimensionality reduction improves computing efficiency while maintaining the inherent structural links in log data, hence enabling more precise anomaly detection. The probabilistic pseudo-labeling mechanism employed in SSLA effectively addresses the scarcity of labeled data, a common limitation in real-world anomaly detection scenarios, by using high-confidence pseudo-labels to augment the training process. This approach mitigates the risks associated with overfitting and enhances the model's generalization capabilities. Furthermore, SSLA's incorporation of differential privacy mechanisms ensures that sensitive data remains secure throughout the anomaly detection process.

Open issues

Despite the promising advancements demonstrated by the proposed SSLA, several open issues and challenges remain that warrant further investigation to enhance the robustness, scalability, and adaptability of anomaly detection systems in cloud and IoT environments. These challenges span algorithmic design, computational efficiency, and real-world applicability, and solving them is key for advancing the field.

Scalability in ultra-large-scale systems

While SSLA shows superior performance on large datasets, the exponential growth of log data in ultra-large-scale distributed systems presents significant scalability challenges. The dynamic and heterogeneous nature of cloud environments necessitate more efficient graph construction and optimization algorithms capable of adapting in real-time without incurring excessive computational overhead. Future research should explore distributed graph processing frameworks and scalable embedding techniques to maintain high throughput and low latency, especially in environments characterized by variable workloads and diverse service dependencies.

Adaptability to evolving threat landscapes

Anomaly detection models, including SSLA, face limitations when detecting zero-day attacks and advanced persistent threats (APTs) that exhibit subtle behavioral deviations. The reliance on historical log data and static similarity metrics may hinder the model's ability to identify novel threats. Incorporating continuous learning mechanisms, such as reinforcement learning and adversarial training, could improve the model's adaptability to emerging threats. Furthermore, self-supervised techniques may enhance the detection of previously unseen anomalies, increasing resilience against sophisticated adversarial strategies.

Balancing privacy and utility

While differential privacy mechanisms within SSLA address data security concerns, striking a balance between privacy preservation and detection accuracy remains an open challenge. Overly stringent privacy constraints may obscure meaningful patterns, reducing detection performance, whereas insufficient privacy measures risk exposing sensitive data. Future work should investigate advanced privacy-preserving techniques, such as federated learning and homomorphic encryption, to ensure that privacy requirements are met without compromising the utility and accuracy of the anomaly detection process.

Explainability and interpretability of anomaly detection

The black-box nature of GCN and semi-supervised learning frameworks complicates the interpretability of detected anomalies. Although SSLA achieves high detection accuracy, providing transparent explanations for anomaly classifications is essential for trust and effective incident response in real-world applications. Future research should focus on developing explainable AI (XAI) techniques to offer comprehensible insights into the model's decision-making process, facilitating more effective collaboration between automated detection systems and human operators.

Cross-domain generalization and transferability

While SSLA demonstrates robust performance on the HDFS and BGL datasets, its generalizability to other domains with different structural and statistical properties remains uncertain. Variations in log formats, data distributions, and anomaly characteristics across domains challenge the transferability of trained models. Research should focus on domain adaptation and transfer learning techniques to maintain high performance in new, unseen environments. Additionally, universal feature representations and cross-domain knowledge transfer could enhance the applicability of anomaly detection models in diverse operational contexts.

Real-time processing and resource constraints

The demand for real-time anomaly detection in IoT and edge computing environments introduces constraints on computational resources, latency, and energy efficiency. While SSLA performs well in cloud environments, adapting it for deployment on resource-constrained devices, such as IoT gateways and edge nodes, presents unique challenges. Techniques like model pruning, quantization, and lightweight graph representations should be explored to reduce computational complexity without sacrificing accuracy. Moreover, integrating edge-cloud collaborative frameworks could optimize resource utilization across heterogeneous computing environments.

Handling imbalanced and noisy data

Anomaly detection systems often struggle with highly imbalanced datasets, where anomalies constitute a small fraction of the data. This imbalance can bias model training towards normal instances, reducing sensitivity to rare but critical anomalies. Additionally, noisy or corrupted log data can degrade performance. While SSLA incorporates probabilistic pseudo-labeling and robust similarity metrics, further advancements are needed to enhance resilience. Future research should explore advanced data augmentation, noise-robust learning algorithms, and hybrid models that combine supervised, semi-supervised, and unsupervised approaches.

Integration with multi-cloud and federated systems

As cloud architectures evolve towards multi-cloud and federated environments, ensuring seamless integration of anomaly detection systems across disparate platforms becomes complex. Differences in data governance, infrastructure heterogeneity, and interoperability standards pose challenges to unified anomaly detection. SSLA's architecture, while effective in single-cloud environments, requires adaptation for multi-cloud scenarios where data locality, latency, and privacy concerns are critical. Future research should focus on decentralized anomaly detection frameworks using federated learning and cross-cloud orchestration to enable collaborative detection without compromising data privacy or system performance.

Conclusion

This research proposed a semi-supervised learning framework for real-time injection detection and anomaly monitoring in cloud-based web applications. The framework combines structured log preprocessing, statistical and temporal feature extraction, and graph-based learning with privacy-preserving mechanisms. A dynamic similarity graph enables effective label propagation using Graph Convolutional Networks, while differential privacy safeguards sensitive information throughout the analysis pipeline. Experimental results on the HDFS and BGL datasets confirmed SSLA's superiority in detection accuracy, latency, throughput, availability, and resource efficiency when compared to state-of-the-art methods. The framework also maintains low CPU and energy consumption, making it suitable for deployment in large-scale and resource-constrained environments.

Although SSLA solves many existing challenges, several directions remain open for further research. Enhancing adaptability to zero-day attacks and evolving threat patterns requires continuous learning and adversarial robustness techniques. Improving generalization across heterogeneous domains, such as IoT and industrial control systems, can extend the model's applicability. More

advanced privacy preserving methods, such as federated learning and homomorphic encryption, may offer stronger guarantees without compromising performance. Finally, integrating explainable AI into SSLA will improve interpretability and trust in anomaly classification decisions. Addressing these challenges will support the development of more secure, scalable, and transparent anomaly detection systems in future cloud and edge computing environments.

Acknowledgements

ACK: This work was supported by a grant of the European Commission, CHIPS Joint Undertaking (G.A. no. 101111977) and of the Ministry of Research, Innovation and Digitization, CNCS/CCCDI -UEFISCDI, project number PN-IV-P8-8.1- PME-2024-0011 Arrowhead flexible Production Value Network (Arrowhead fPVN)", within PNCDI IV, and partially supported by a grant of the Ministry of Research, Innovation and Digitization, CNCS/CCCDI -UEFISCDI, project number PN-IV-P8-8.1-PRE-HE-ORG-2023-0063 Arrowhead fPVN PI", within PNCDI IV and supported by a grant of the European Commission no. 101183162 (ANTIDOTE project) and of the Ministry of Research, Innovation and Digitization, CNCS/CCCDI -UEFISCDI, project number PN-IV-P8-8.1-PREHE-ORG-2024-0236, within PNCDI IV. The APC of this article was funded National University of Science and Technology POLITEHNICA Bucharest through the 'PubArt' Programme.

Authors' contributions

S.S.S. conceptualized the study, designed the methodology, and wrote the main manuscript text. B.A. contributed to data collection, preprocessing, and implementation of the semi-supervised framework. O.F. and S.H. provided critical insights on cloud security mechanisms and supervised the research. S.S.S. and B.A. conducted the experiments and analyzed the results. All authors reviewed and approved the final manuscript.

Funding

This work was supported by a grant of the European Commission, CHIPS Joint Undertaking (G.A. no. 101111977) and of the Ministry of Research, Innovation and Digitization, CNCS/CCCDI -UEFISCDI, project number PN-IV-P8-8.1- PME-2024-0011 Arrowhead flexible Production Value Network (Arrowhead fPVN)", within PNCDI IV, and partially supported by a grant of the Ministry of Research, Innovation and Digitization, CNCS/CCCDI -UEFISCDI, project number PN-IV-P8-8.1-PRE-HE-ORG-2023-0063 Arrowhead fPVN PI", within PNCDI IV and supported by a grant of the European Commission no. 101183162 (ANTIDOTE project) and of the Ministry of Research, Innovation and Digitization, CNCS/CCCDI -UEFISCDI, project number PN-IV-P8-8.1-PRE-HE-ORG-2024-0236, within PNCDI IV.

Data availability

No datasets were generated or analysed during the current study.

Declarations

Ethics approval and consent to participate

Not applicable.

Consent for publication

All authors confirm that neither the article nor any parts of its content are currently under consideration or published in another journal. The authors agree to publication in the journal.

Competing interests

The authors declare no competing interests.

Received: 10 February 2025 / Accepted: 11 June 2025

Published online: 16 July 2025

References

1. Arasteh B, Arasteh K, Kiani F, Sefati SS, Fratu O, Halunga S, Tirkolaee EB (2024) A bioinspired test generation method using discretized and modified Bat optimization algorithm. *Mathematics* 12(2):186
2. Shiraz M, Gani A, Khokhar RH, Buyya R (2012) A review on distributed application processing frameworks in smart mobile devices for mobile cloud computing. *IEEE Commun Surv Tutor* 15(3):1294–1313
3. Verma R, Rane D (2024) Service-Oriented Computing: Challenges, Benefits, and Emerging Trends, *Soft Computing Principles and Integration for Real-Time Service-Oriented Computing*, pp. 65–82
4. Singh S, Jeong Y-S, Park JH (2016) A survey on cloud computing security: issues, threats, and solutions. *J Netw Comput Appl* 75:200–222
5. Habibzadeh H, Nussbaum BH, Anjomshoa F, Kantarci B, Soyata T (2019) A survey on cybersecurity, data privacy, and policy issues in cyber-physical system deployments in smart cities. *Sustainable Cities Soc* 50:101660
6. Pawlik Ł (2025) Google cloud vs. Azure: sentiment analysis accuracy for Polish and english across content types. *J Cloud Comput* 14(1):17
7. Arasteh B, Bouyer A, Sefati SS, Craciunescu R (2024) Effective SQL Injection Detection: A Fusion of Binary Olympiad Optimizer and Classification Algorithm. *Mathematics* 12(18):2917
8. Gupta S, Gupta BB (2017) Cross-Site scripting (XSS) attacks and defense mechanisms: classification and state-of-the-art. *Int J Syst Assur Eng Manage* 8:512–530
9. Aslan Ö, Aktuğ SS, Ozkan-Okay M, Yilmaz AA, Akin E (2023) A comprehensive review of cyber security vulnerabilities, threats, attacks, and solutions. *Electronics* 12(6):1333
10. Cao J, Zhang C, Qi P, Hu K (2025) Utility-driven virtual machine allocation in edge cloud environments using a partheno-genetic algorithm. *J Cloud Comput* 14(1):1–15
11. Domingo-Ferrer J, Farras O, Ribes-González J, Sánchez D (2019) Privacy-preserving cloud computing on sensitive data: A survey of methods, products and challenges. *Comput Commun* 140:38–60
12. Zhang S, Tong H, Xu J, Maciejewski R (2019) Graph convolutional networks: a comprehensive review. *Comput Social Networks* 6(1):1–23
13. Almansouri HT, Masmoudi Y Hadoop distributed file system for big data analysis, in (2019) 4th World Conference on Complex Systems (WCCS), 2019: IEEE, pp. 1–5
14. Meghana C, Kariyappa B Supervised Learning for Log-Based Anomaly Detection, in (2024) 8th International Conference on Computational System and Information Technology for Sustainable Solutions (CSITSS), 2024: IEEE, pp. 1–4
15. Sefati SS, Craciunescu R, Arasteh B, Halunga S, Fratu O, Tal I (2024) Cybersecurity in a Scalable Smart City Framework Using Blockchain and Federated Learning for Internet of Things (IoT). *Smart Cities* 7(5):2802–2841
16. Paul A, Sharma V, Olukoya O (2024) SQL injection attack: detection, prioritization & prevention. *J Inform Secur Appl* 85:103871
17. Farahani A, Delkhosh H, Seifi H, Azimi M (2024) A new bi-level model for the false data injection attack on real-time electricity market considering uncertainties. *Comput Electr Eng* 118:109468
18. Deepa G, Thilagam PS (2016) Securing web applications from injection and logic vulnerabilities: approaches and challenges. *Inf Softw Technol* 74:160–180
19. Lu J, Lan J, Huang Y, Song M, Liu X (2023) Anti-attack intrusion detection model based on MPNN and traffic Spatiotemporal characteristics. *J Grid Comput* 21(4):60
20. Khan IA, Pi D, Kamal S, Alsuaibani M, Alshammari BM. Federated-Boosting: A Distributed and Dynamic Boosting-Powered Cyber-Attack Detection Scheme for Security and Privacy of Consumer IoT. In *IEEE Trans Consum Electron*. <https://doi.org/10.1109/TCE.2024.3499942>
21. Kaushal P, Kaur P (2025) Cyber security techniques architecture and design: Cyber-Attacks detection and prevention. *Advancing cyber security through quantum cryptography*. IGI Global, pp 231–258
22. Haq AU, Sefati SS, Nawaz SJ, Mihovska A, Beliatas MJ (2025) Need of UAVs and Physical Layer Security in Next-Generation Non-Terrestrial Wireless Networks: Potential Challenges and Open Issues. In *IEEE Open J of Veh Technol* 6:544–595. <https://doi.org/10.1109/OJVT.2025.3525781>.
23. Van Engelen JE, Hoos HH (2020) A survey on semi-supervised learning. *Mach Learn* 109(2):373–440
24. Pise NN, Kulkarni P A survey of semi-supervised learning methods, in (2008) *International conference on computational intelligence and security*, 2008, vol. 2: IEEE, pp. 30–34

25. Khan IA et al (2024) Fed-inforce-fusion: A federated reinforcement-based fusion model for security and privacy protection of IoT networks against cyber-attacks. *Inform Fusion* 101:102002
26. Nassif AB, Talib MA, Nasir Q, Dakalbab FM (2021) Machine learning for anomaly detection: A systematic review. *Ieee Access* 9:78658–78700
27. Palmieri F, Fiore U (2010) Network anomaly detection through nonlinear analysis. *Computers Secur* 29(7):737–755
28. Sefati SS, Halunga S (2022) Mobile sink assisted data gathering for URLLC in IoT using a fuzzy logic system, 2022 IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom). Sofia 379–384. <https://doi.org/10.1109/BlackSeaCom54372.2022.9858268>
29. Pimenta Rodrigues GA et al (2017) Cybersecurity and network forensics: analysis of malicious traffic towards a honeynet with deep packet inspection. *Appl Sci* 7(10):1082
30. Lagraa S, Husák M, Seba H, Vuppala S, State R, Ouedraogo M (2024) A review on graph-based approaches for network security monitoring and botnet detection. *Int J Inf Secur* 23(1):119–140
31. Ghadermazi J, Shah A, Bastian ND (2025) Towards Real-Time Network Intrusion Detection With Image-Based Sequential Packets Representation. In *IEEE Transactions on Big Data* 11(1):157–173. <https://doi.org/10.1109/TBDATA.2024.3403394>
32. Eswaran S, Srinivasan A, Honnavalli P (2021) A threshold-based, real-time analysis in early detection of endpoint anomalies using SIEM expertise. *Network Security* 4:7–16
33. Demertzi V, Demertzis S, Demertzis K (2023) An Overview of Privacy Dimensions on the Industrial Internet of Things (IIoT). *Algorithms* 16(8):378
34. Bezas K, Filippidou F (2023) Comparative analysis of open source security information & event management systems (SIEMs). *Indonesian J Comput Sci* 12(2):443–468
35. Noman HA, Abu-Sharkh OM (2023) Code injection attacks in wireless-based Internet of Things (IIoT): A comprehensive review and practical implementations. *Sensors* 23(13):6067
36. Theodoropoulos T et al (2023) Security in Cloud-Native services: A survey. *J Cybersecur Priv* 3(4):758–793
37. Alam S, Alam Y, Cui S, Akujobi C (2023) Data-driven network analysis for anomaly traffic detection. *Sensors* 23(19):8174
38. Zeng X, Zhuo Y, Liao T, Guo J (2025) Cloud-GAN: cloud generation adversarial networks for anomaly detection. *Pattern Recogn* 157:110866
39. Mohajerani S, Saeedi P (2021) Cloud and cloud shadow segmentation for remote sensing imagery via filtered Jaccard loss function and parametric augmentation. *IEEE J Sel Top Appl Earth Observations Remote Sens* 14:4254–4266
40. Chu G, et al (2025) Anomaly Detection on Interleaved Log Data With Semantic Association Mining on Log-Entity Graph. In *IEEE Transactions on Software Engineering* 51(2):581–594. <https://doi.org/10.1109/TSE.2025.3527856>
41. Zhang X, Yan W, Li H (2024) False data injection attacks detection and state restoration based on power system interval dynamic state Estimation. *Comput Electr Eng* 118:109347
42. Gøttcke JM, Zimek A, Campello RJ (2025) Bayesian label distribution propagation: A semi-supervised probabilistic k nearest neighbor classifier. *Inform Syst* 129:102507
43. Song Y et al (2025) A multi-source log semantic analysis-based attack investigation approach. *Computers Secur* 150:104303
44. Chen J, Ying R (2023) Tempme: towards the explainability of Temporal graph neural networks via motif discovery. *Adv Neural Inf Process Syst* 36:29005–29028
45. Sefati SS, Fartu O, Nor AM, Halunga S (2024) Enhancing Internet of Things security and efficiency: Anomaly detection via proof of stake blockchain techniques, in 2024 International Conference on Artificial Intelligence in Information and Communication (ICAIIIC); IEEE, pp. 591–595
46. Leushuis RM (2025) Probabilistic forecasting with VAR-VAE: Advancing time series forecasting under uncertainty. *Info Sci* 713. p. 122184
47. Jeffrey N, Tan Q, Villar JR (2023) A review of anomaly detection strategies to detect threats to cyber-physical systems. *Electronics* 12(15):3283
48. Habeeb RAA, Nasaruddin F, Gani A, Hashem IAT, Ahmed E, Imran M (2019) Real-time big data processing for anomaly detection: A survey. *Int J Inf Manag* 45:289–307
49. Khorshed MT, Ali AS, Wasimi SA (2012) A survey on gaps, threat remediation challenges and some thoughts for proactive attack detection in cloud computing. *Future Generation Comput Syst* 28(6):833–851
50. Sefati SS, Nor AM, Arasteh B, Craciunescu R, Comsa C-R (2025) A probabilistic approach to load balancing in Multi-Cloud environments via machine learning and optimization algorithms. *J Grid Comput* 23(2):1–36
51. Adhikari D, Jiang W, Zhan J, Rawat DB, Bhattarai A (2024) Recent advances in anomaly detection in internet of things: status, challenges, and perspectives. *Comput Sci Rev* 54:100665
52. Sefati SS, Halunga S (2022) Data forwarding to Fog with guaranteed fault tolerance in Internet of Things (IIoT). 2022 14th International Conference on Communications (COMM). Bucharest 1–5. <https://doi.org/10.1109/COMM544.2022.9817179>
53. Liu J, Huang Y, Deng C, Zhang L, Chen C, Li K (2025) Efficient resource allocation algorithm for maximizing operator profit in 5G edge computing network. *J Grid Comput* 23(1):1–18
54. Agos Jawaddi SN, Ismail A, Sulaiman MS, Cardellini V (2025) Analyzing Energy-Efficient and Kubernetes-Based autoscaling of microservices using probabilistic model checking. *J Grid Comput* 23(1):3
55. Studiawan H, Soheli F, Payne C (2020) Anomaly detection in operating system logs with deep learning-based sentiment analysis. *IEEE Trans Dependable Secur Comput* 18(5):2136–2148
56. Xu Z, Wang Z, Xu J, Shi H, Zhao H (2024) Enhancing log anomaly detection with semantic embedding and integrated neural network innovations. *Computers Mater Continua* 80:3
57. Mir AA, Zuhairi MF, Musa SM (2023) Graph anomaly detection with graph convolutional networks. *Int J Adv Comput Sci Appl* 14:11

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.